

**Fachhochschule
Dortmund**

University of Applied Sciences and Arts

MASTERSTUDIENARBEIT

*Untersuchung und Charakterisierung der Open-
Source-Toolchain (Yosys / Nextpnr) für das
GateMate-FPGA: Analyse von Netzlisten und
Zwischenformaten*

Vorgelegt von:

Herrn Simo Gueno Yvan Valder

Erstprüfer: Prof. Dr. Hugues Tchouankem

Betreuer: Prof. Dr. Jens Rettkowski

Datum: Oktober 2026

Kurzzusammenfassung

Die vorliegende Masterstudienarbeit befasst sich mit der Untersuchung und Charakterisierung der Open-Source-Toolchain, bestehend aus Yosys für die Logiksynthese und Nextpnr für das Place & Route (P&R), am Beispiel des GateMate-FPGAs der Cologne Chip AG. Während traditionelle FPGA-Entwicklungsumgebungen auf proprietären, geschlossenen Werkzeugen basieren, bieten Open-Source-EDA-Frameworks (Electronic Design Automation) maximale Transparenz, erfordern jedoch ein tiefes Verständnis ihrer internen Datenstrukturen, Zwischenformate und Netzlisten.

Das Hauptziel dieser Arbeit ist die systematische Analyse des vollständigen Entwurfsflusses anhand eines repräsentativen Verilog-Testdesigns. Dabei werden insbesondere die Struktur der post-synthetisierten Netzlisten, die Handhabung und Weitergabe von Metainformationen (wie Attributen und Constraints) durch Yosys sowie die konkreten Dateiformate und Repräsentationen nach dem erfolgreichen P&R-Prozess durch Nextpnr untersucht und dokumentiert. Die Ergebnisse schaffen ein fundiertes technisches Verständnis der internen Toolchain-Mechanismen und bieten eine wertvolle Basis für künftige FPGA-Entwicklungsprojekte im Open-Source-Bereich.

Abstract

This Thesis investigates and characterizes the open-source EDA toolchain specifically Yosys for logic synthesis and Nextpnr for place and route (P&R) targeting the GateMate FPGA by Cologne Chip AG. While traditional FPGA development flows rely on proprietary, closed-source toolchains, open-source alternatives provide high transparency but require a thorough understanding of their internal data structures, intermediate formats, and netlists.

The primary objective of this work is to systematically analyze and document the complete design flow using a representative Verilog test design. Key areas of investigation include the structure of post-synthesis netlists, the handling and retention of metadata and constraints within Yosys, and the specific file formats and internal representations generated after Nextpnr's P&R process. The insights gained provide a comprehensive understanding of the toolchain's internal mechanisms, serving as a solid foundation for future open-source FPGA development and optimization workflows.

Inhaltsverzeichnis

Kurzzusammenfassung	2
Inhaltsverzeichnis	3
1 Einleitung	6
2 GateMate-FPGA-Grundlagen	7
3 Theorie: RTLIL, Yosys und nextpnr	8
3.1 Abstraktionsebenen	8
3.2 Vom Text zum Silizium	10
3.2.1 Der AST	11
3.2.2 RTLIL-Generierung nach Knotentyp	11
3.2.3 RTLIL-Objekte	12
3.2.4 Verilog-Prozess	12
3.2.5 Die Zellbibliothek des GateMates	14
3.2.5.1 Multiplexer	14
3.2.5.2 Look-Up-Tables	14
3.2.5.3 Register	15
3.2.5.4 Arithmetik	15
3.2.5.5 Soft-DSP	15
3.3 Yosys	16
3.3.2 Die einzelnen Stationen	16
3.3.2.1 Read_verilog	16
3.3.2.2 hierarchy -ckeck -top	16
3.3.2.3 proc	16
3.3.2.4 opt	17
3.3.2.5 fsm und memory	18
3.3.2.6 techmap	18
3.3.2.7 synth_gatemate	18
3.3.3 Syntheseskripte	18

3.3.3.1 manuelle Syntheseskripte	18
3.3.3.2 generische Syntheseskripte	18
3.3.3.3 GateMate spezifische Syntheseskripte	19
3.3.3.3 Netliste in JSON-Format	19
3.4 nextpnr-himbaechel	21
3.4.1 Packing.....	22
3.4.2 Placement	22
3.4.3 Routing.....	22
4 Praktische Umsetzung: Volladdierer	23
4.1 Zielsetzung und Versuchsaufbau	23
4.1.1 Variante A — Datenflussbeschreibung.....	23
4.1.2 Variante B — Prozessbeschreibung	23
4.1.3 Variante C — Prozessbeschreibung mit case-Verzweigung	23
4.2 Analyse des Datenflusses (Variante A).....	26
4.3 Analyse des Prozesspfades (Variante B).....	29
4.4 Analyse des Prozess- und Case-Pfades (Variante C)	29
4.5 Place & Route	33
4.6 Bewertung der Ergebnisse	34
4.6.1 Yosys bildet die geschriebene Struktur ab, nicht die boolesche Bedeutung ..	34
4.6.2 Die Netzliste ist keine Ressourcenmessung	34
4.6.3 Die Erkennung als ROM skaliert nicht beliebig.....	35
Fazit.....	36
Litteraturverzeichnis	37
Normen, Datenblätter und Herstellerdokumentation (Cologne Chip AG)	37
Wissenschaftliche Arbeiten und Literatur	37
Software-Dokumentation, Frameworks und Open-Source-Distributionen	37
Online- und Videoquellen	38
Abkürzungsverzeichnis	38
Abbildungverzeichnis.....	40
Tabellenverzeichnis	41

Appendix	42
Appendix A Installation und Konfiguration von Yosys und Nextpnr	42
Appendix A.1 Yosys installieren: Die Open-Source-Synthesesuite einrichten	42
Appendix A.2 Nextpnr einrichten: Das Platzierungs- und Routing-Tool (Place & Route) konfigurieren	43
Appendix A.3 Project Peppercorn (GateMate-Backend)	44
Appendix B GateMate FPGA Open-Source-Toolchain & Workflow [37]	45
Appendix B1 Komponenten der OSS CAD Suite	45
Appendix B.2 Synthese mit Yosys (synth_gatemate)	46
Appendix B.3 Implementierung: Place & Route mit nextpnr-himbaechel	47
Appendix B.4 Bitstream-Generierung mit gmpack	49
Appendix B.5 Hardware-Konfiguration mit openFPGALoader	50
Appendix C Quellen code	50
Appendix C.1 Voll_addierer ohne Process	50
Appendix C.2 Voll_addierer mit Process	51
Appendix C.3 Voll_addierer mit Process und case	51
Appendix C.4 Yosyskript für die Analyse der Zwischenschritte	52
Appendix D Simulation der Variante A/B und C	55
Appendix D.1 Simulation der Synthesenergebnisse der Variante A/B mit logic.ly	55
Appendix D.2 Simulation der Synthesenergebnisse der Variante C mit simulator.io [9]	61

1 Einleitung

In der modernen digitalen Systementwicklung stellen FPGAs eine zentrale Technologie dar, um rechenintensive Algorithmen, Signalverarbeitungs- und eingebettete Systeme flexibel und effizient zu realisieren. Traditionell basierte der Entwurfs- und Syntheseprozess von FPGAs (Electronic Design Automation, EDA) fast ausschließlich auf geschlossenen, proprietären Toolchains der jeweiligen Chiphersteller. Diese geschlossenen Ökosysteme bieten zwar ausgereifte Arbeitsabläufe, schränken jedoch die **Transparenz**, die **Modifizierbarkeit** und die **Integration** in automatisierte Continuous-Integration-Pipelines stark ein.

In den letzten Jahren hat sich durch den Aufstieg von Open-Source-EDA-Frameworks wie **Yosys** (für die Logiksynthese) und **Nextpnr** (für das Place & Route) ein Paradigmenwechsel vollzogen. Diese Werkzeuge ermöglichen einen offenen, plattformunabhängigen Zugang zur Hardware-Synthese. Ein bemerkenswerter Vertreter moderner, flexibler Halbleiterarchitekturen ist das **GateMate-FPGA** der Cologne Chip AG, das durch eine innovative Architektur und zunehmende Unterstützung in Open-Source-Toolchains gekennzeichnet ist. Die Kombination aus Yosys, Nextpnr und dem GateMate-Zielsystem eröffnet neue Perspektiven für die transparente und anpassbare Hardwareentwicklung, wirft jedoch gleichzeitig Fragen bezüglich der internen Datenstrukturen und Verarbeitungsschritte auf.

2 GateMate-FPGA-Grundlagen

Die GateMate FPGAs sind architektonisch deutlich von den verbreiteten FPGAs unterschiedlich, indem die auf einer homogenen, feingranularen Matrixarchitektur ohne dedizierte, feste DSP-Blöcke basieren (siehe Tabelle 1: **Ressourcen des CCGM1A1** (Evaluation Board)). Sämtliche arithmetischen Operationen werden direkt in die Logikmatrix (CPE) abgebildet. (siehe Abbildung 1:)

Tabelle 1: **Ressourcen des CCGM1A1** (Evaluation Board)

Ressource / Merkmal	Spezifikation (CCGM1A1)	Funktionale Besonderheit
Cologne Programmable Elements (CPE)	20.480	Universelle Basiszellen der Logikmatrix
Register (Flip-Flops / Latches)	40.960	Exakt 2 Speicherglieder pro CPE (CC_DFF, CC_DLT)
Logikkapazität (LUT4-Äquivalent)	bis zu 40.960	Realisiert über Dual-LUT4-Konfiguration pro CPE
Block-RAM (DPSRAM)	1.280 kbit	Echte Dual-Port-SRAM-Blöcke (20 oder 40 kbit konfigurierbar)
Feste DSP-Blöcke (Hard-IP)	0	Arithmetik erfolgt über CPE-Verschaltung (CC_MULT)
Takt- und I/O-Ressourcen	4 CC_PLL, SerDes	SerDes bis zu 5 Gbit/s, GPIO, SPI, JTAG

Jedes CPE enthält einen LUT-Baum mit acht Eingängen, zwei Flip-Flops, schnelle Signalpfade zu den Nachbarelementen und zwei Ausgänge (siehe Abbildung 1:). Es lässt sich konfigurieren als:

- ein LUT-Baum mit 8 Eingängen **oder** zwei unabhängige mit je 4 Eingängen,
- Multiplexerfunktion mit 4 Eingängen (6 Eingänge gesamt),
- 1- oder 2-Bit-Volladdierer, kaskadierbar,
- 2×2-Bit-Multiplizierer, kaskadierbar.

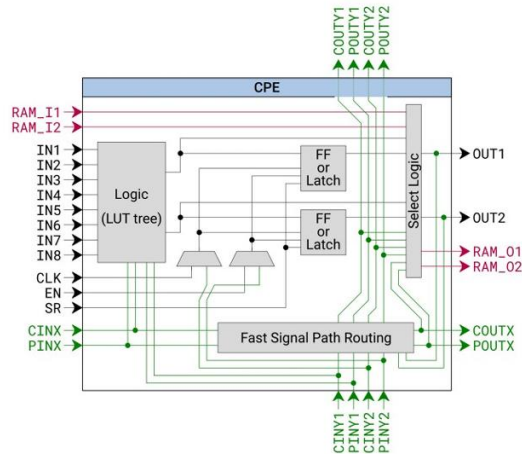


Abbildung 1: CPE-Logikelement[2]

3 Theorie: RTLIL, Yosys und nextpnr

In diesem Kapitel wird beschrieben, was Yosys tatsächlich im Speicher Schritt für Schritt erzeugt und wie nextpnr die Zellen platziert und routet. Die Installation der Toolchains wird in [Installation und Konfiguration von Yosys und Nextpnr](#) beschrieben.

3.1 Abstraktionsebenen

Ein digitaler Entwurf lässt sich auf sechs Ebenen beschreiben, von einer hohen Abstraktionsebene zu einer niedrigen, der sogenannten Silizium-Ebene (siehe Tabelle 2: Abstraktionsebenen der FPGA-Entwurf). Die Einordnung ist notwendig, um den Zuständigkeitsbereich von Yosys abzugrenzen.

Tabelle 2: Abstraktionsebenen der FPGA-Entwurf

Nr.	Ebene	Beschreibung	Beispiel	Werkzeug	
1	System	Blöcke und Datenflüsse	SystemC, Blockdiagramm	Simulink, Werkzeuge	UML-

2	Algorithmisch	Berechnung ohne Takt	for (i=0;i<8;i++) s += a[i]*b[i]; (C/C++, Python ..)	• Vitis HLS, • Intel HLS, • HLS4ML
3	RTL	Register und Transfers, Takt explizit	always @(posedge clk) acc <= acc + a*b;	Yosys (Eingang)
4	Logikgatter	Boolesche Funktionen, 1 Bit	\$_AND_	Yosys
5	Physikalisches Gatter	Bausteinspezifische Zellen	CC_LUT2	Yosys (Ausgang)
6	Transistor	NMOS/PMOS	SPICE-Netzliste	Cadence Virtuoso

Yosys deckt die Ebenen 3 bis 5 ab. Die Ebene 2 ist Aufgabe der High-Level-Synthese und die Ebene 6 ist beim FPGA werkseitig festgelegt.

Aus technischer Sicht würden die Dumps es ermöglichen, festzustellen, auf welcher Abstraktionsebene sich das Programm befindet, wobei die Form oder die Unterstriche den Übergang zur Ebene der Logikgatter markieren, während das Fehlen des anfänglichen **\$**-Zeichens den Übergang zur Siliziumebene anzeigen würde. (siehe Tabelle 3: Erkennungsmerkmal im Dump)

Tabelle 3: Erkennungsmerkmal im Dump

Form	Ebene	Breite	Parameter
\$and	RTL-Operator	n Bit	A_WIDTH, A_SIGNED, ...
<u>\$_AND_</u>	Logikgatter	1 Bit	keine
CC_LUT2	Physikalisch	1 Bit	INIT

3.2 Vom Text zum Silizium

Yosys übersetzt nicht Wort für Wort, sondern über eine Zwischenstruktur — vergleichbar der grammatikalischen Analyse beim Übersetzen eines Textes. Die Abbildung 2: Detaillierter Workflow von einem Verilog-Code bis zum Silizium unter Verwendung von Yosys für die Synthese und NextPnr für das Routing und die Platzierung zeigt im Detail alle Schritte, die erforderlich sind, um eine FPGA-Hardware anhand eines vorgegebenen Textcodes physikalisch zu beschreiben.

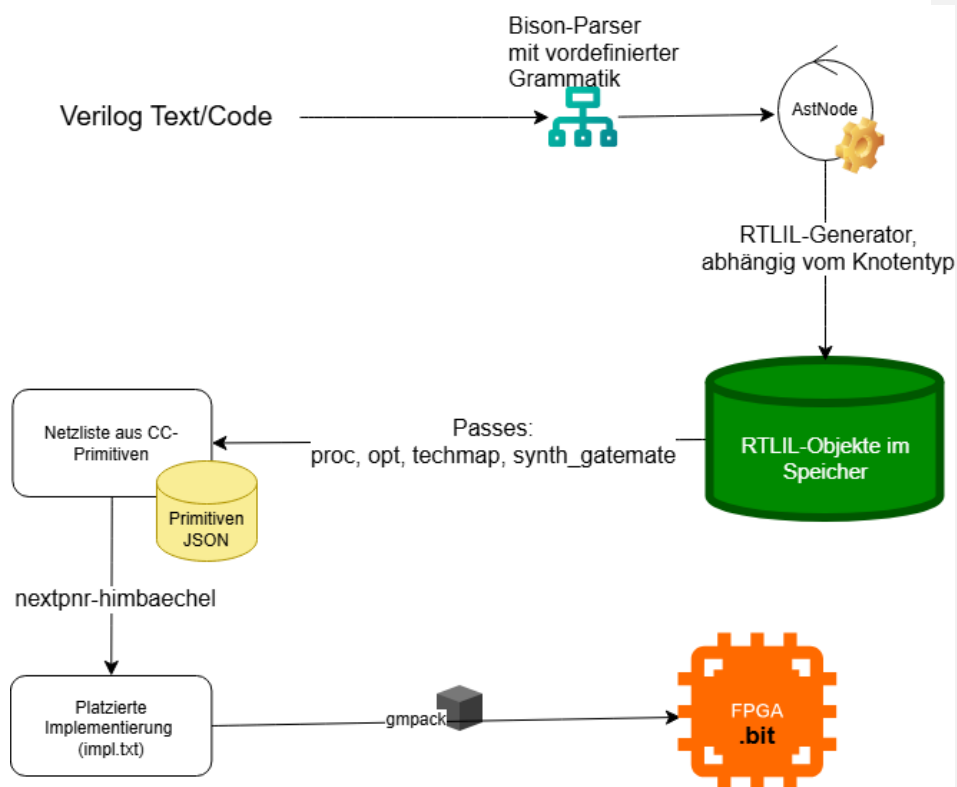


Abbildung 2: Detaillierter Workflow von einem Verilog-Code bis zum Silizium unter Verwendung von Yosys für die Synthese und NextPnr für das Routing und die Platzierung

3.2.1 Der AST

Ein AST ist ein Speicher befindlicher Baum, der die grammatikalische Struktur und die Bedeutung eines Quelltexts darstellt. Daran erzeugt der Parser (bzw. der Bison-Parser in Verbindung mit Flex) aus dem Quelltext AstNode-Objekte. Der Baum entsteht allein durch das Anfügen von Knoten als Kinder, wobei ein Stapel als Hilfsobjekt dient.

Für `assign y = a & b;` ergibt sich:

AST_ASSIGN

└─ AST_IDENTIFIER "y"

└─ AST_BIT_AND

└─ AST_IDENTIFIER "a"

└─ AST_IDENTIFIER "b"

Der AST ist **keine Hardware-Abstraktionsebene**, sondern eine Repräsentation der Sprache. Er erscheint daher in keinem der sechs Ebenen aus **3.1 Abstraktionsebenen** und in keinem Dump.

3.2.2 RTLIL-Generierung nach Knotentyp

Die Analyse des Yosys-Quellcodes durch Parnow [37] zeigt, dass die Erzeugung von RTLIL vom Typ des AST-Knotens abhängt. (siehe Tabelle 4: Beispiel für die Zuordnung zwischen einem AST-Knotentyp und einem RTLIL-Objekt)

Tabelle 4: Beispiel für die Zuordnung zwischen einem AST-Knotentyp und einem RTLIL-Objekt

Knotentyp	Behandlung	Ergebnis
AST_WIRE	direkte Übernahme	RTLIL::Wire
Operationen (AST_BIT_AND, AST_ADD)	direkte Umwandlung	RTLIL::Cell (\$and, \$add)
AST_ALWAYS	zusätzlicher Prozessgenerator	RTLIL::Process

Da Operationsknoten unmittelbar in Zellen überführt werden, die anschließend durch Technology Mapping auf reale Gatter abgebildet werden, lässt sich schließen, dass **bereits bei der RTLIL-Generierung erste Syntheseschritte erfolgen** [37].

3.2.3 RTLIL-Objekte

Das Textformat des Dumps ist die lesbare Darstellung einer C++-Klassenhierarchie [38]. In C++ ist RTLIL jedoch keine eigenständige Klasse, sondern ein Namensraum (namespace), der die interne Struktur einer elektronischen Schaltung darstellt. (siehe Tabelle 5: Zuweisung eines bestimmten Befehls in Textform in der Klasse RTLIL C++)

Tabelle 5: Zuweisung eines bestimmten Befehls in Textform in der Klasse RTLIL C++

Im Dump	C++-Klasse	Inhalt
process	RTLIL::Process	ein CaseRule als Wurzel, ein Vektor SyncRule
switch	RTLIL::SwitchRule	auszuwertende SigSpec, Liste von CaseRule
case	RTLIL::CaseRule	compare, actions, switches
sync	RTLIL::SyncRule	Signal, SyncType, Aktionen
connect	RTLIL::SigSig	Paar aus zwei SigSpec
1'0	RTLIL::SigSpec	SigChunk und SigBit mit RTLIL::State

Der SyncType entscheidet über die erzeugte Hardware:

- STa (sync always) führt zu rein kombinatorischer Logik,
- STp (sync posedge) zu Flip-Flops.

3.2.4 Verilog-Prozess

Ein Verilog-Prozess beschreibt einen parallel ablaufenden Verhaltensblock, der die Aktivität im Zeitverlauf modelliert. Um dessen Auswirkungen auf die Synthese zu untersuchen, wurde dieselbe logische Funktion in zwei verschiedenen Notationen beschrieben und der Zustand nach dem Auslesen verglichen.

Tabelle 6: Syntheseergebnisse zwei verschiedenen Notationen

	Variante A	Variante B
Verilog-Code	<code>assign out = a & b;</code>	<code>always @ (*) begin</code> <code>if (a) out = b;</code> <code>else out = 1'b0;</code> <code>end</code>
AST-Knoten	AST_ASSIGN mit AST_BIT_AND	AST_ALWAYS
Behandlung	direkte Umwandlung	Prozessgenerator
Stat-Befehl nach read_verilog	1 cell, \$and	0 cells, 1 process
Nach proc- Befehl	unverändert	1 cell, \$mux

Das Ergebnis der Variante B ist ein `$mux` und kein `$and`, da Yosys die vom Entwickler geschriebene Struktur wiedergibt und nicht deren boolesche Bedeutung. Die Äquivalenz `a & b ≡ a ? b : 0` wird erst im Rahmen der nachgelagerten Optimierung und der LUT-Zuordnung aufgelöst. Dennoch führen beide Varianten auf einem GateMate-FPGA zur gleichen LUT.

Die **Proc**-Durchläufe dokumentieren diesen Prozess. `proc_rmdead` stellt fest, dass die beiden „case“-Zweige alle Werte abdecken, und aktiviert das Attribut `full_case`. - `proc_dlatch` meldet daraufhin explizit „No latch inferred“. Die genaue Beschreibung des Makro-Proc wird im Yosys-Teil (siehe **3.3 Yosys**) dieses Berichts erläutert.

Wird der „else“-Zweig entfernt, muss der Wert bei `a = 0` unverändert bleiben – Yosys muss dann zwingend einen `$d latch` generieren. An dieser Stelle entsteht ein unbeabsichtigter Latch, der vermieden werden muss.

3.2.5 Die Zellbibliothek des GateMates

Die Primitiven des GateMates lassen sich auf fünf Grundfunktionen zurückführen. Die Tabelle 8: Darstellung von Verilog-Quellcode in einer physikalischen FPGA-Zelle zeigt die Darstellung bestimmter Grundfunktionen in einer physikalischen FPGA-Zelle. Die genauen Namen und die Liste der Zellen befindet sich in der Datei „share/yosys/gatemate/cells_sim.v“. Diese Datei beschreibt das Verhalten jeder Primitive in lesbarem Verilog und wird von „synth_gatemate“ im allerersten Schritt geladen.

3.2.5.1 Multiplexer

Ein Multiplexer wählt ein Eingangssignal aus, ohne es zwischenzuspeichern. Das CPE realisiert diese Funktion nativ und seine Bibliothek umfasst ausschließlich die Zellen CC_MX2 und CC_MX4. Größere Strukturen wie MUX8 existieren nicht als Einzelzelle und werden kaskadiert — daher ist `-nomx8` (bzw. kein MUX 8) zwingend.

3.2.5.2 Look-Up-Tables

Die LUT sind Multiplexer, deren Dateneingänge mit statischen SRAM-Konfigurationszellen verbunden sind. Folglich existieren in der Logikmatrix keine physischen Gatter: `assign s = a & b;` erzeugt eine LUT2 mit INIT = 4'b1000 (siehe Tabelle 7: Beispiel Konfigurationswerte für den LUT). GateMate verwendet LUT2 in Baumstruktur (CC_L2T4 aus drei LUT2, CC_L2T5), die kombiniert ein CPE mit acht Eingängen belegen.

Tabelle 7: Beispiel Konfigurationswerte für den LUT

LUT-Eingang Code	Zelle/Funktion
4'1010	Durchleitung
4'1000	AND
4'1110	OR
4'0110	XOR

3.2.5.3 Register

Jedes CPE enthält zwei physikalische Speicherglieder, wahlweise taktflankengetriggert (CC_DFF) oder pegelgesteuert (CC_DLT), mit synchronem oder asynchronem Reset und Clock-Enable. Ein Latch verursacht keine zusätzlichen Siliziumkosten, erschwert jedoch die statische Zeitanalyse.

3.2.5.4 Arithmetik

Es handelt sich um ein kaskadierbarer Volladdierer mit schnellem Übertragungspfad (CC_ADDF).

3.2.5.5 Soft-DSP

Da keine festen DSP-Slices integriert sind, wird Arithmetik generisch synthetisiert. Die Makro-Primitive CC_MULT bildet vorzeichenbehaftete Multiplizierer beliebiger Wortbreite auf die 2×2-Multiplizierer- und Addiererstufen der CPEs ab, optional gepipelined über CC_DFF.

Tabelle 8: Darstellung von Verilog-Quellcode in einer physikalischen FPGA-Zelle

Quelltext	RTL-Zelle	Gatter Zelle) (Logische	Physikalische Zelle
a & b	\$and	\$_AND_	CC_LUT2, CC_L2T4
a ? b : c	\$mux	\$_MUX_	CC_MX2, CC_MX4
always @(posedge)	\$dff	\$_DFFE_????_	CC_DFF
unvollständiges always @(*)	\$dlatch	\$_DLATCH_???_	CC_DLT
a + b	\$add, \$alu	—	CC_ADDF
a * b	\$mul	—	CC_MULT
Speicherfeld	\$mem_v2	—	CC_BRAM_20K/40K

3.3 Yosys

Yosys (*Yosys Open SYnthesis Suite*) ist ein quelloffenes Framework für die RTL-Synthese. Es liest eine Beschreibung in Verilog oder SystemVerilog ein und erzeugt daraus mithilfe von Pässen eine Netzliste. Jeder Pass oder Durchlauf verändert das RTLIL **an Ort und Stelle**, sodass keine Zwischendateien erstellt werden. Ein Dump ist daher lediglich eine Momentaufnahme des Speicherzustands. Der Ablauf ist somit keine Kette von Übersetzungen zwischen Formaten, sondern eine Folge von Umformungen an einer einzigen Datenstruktur.

3.3.2 Die einzelnen Stationen

3.3.2.1 Read_verilog

read_verilog führt zwei Umwandlungen in einem Befehl aus: Text zu AST, AST zu RTLIL. Eine kontinuierliche Zuweisung wird unmittelbar zu einer Zelle, ein `always`-Block dagegen zu einem `process`. Ob die Zeile `processes` in der Statistik erscheint, zeigt daher an, welcher der beiden Wege beschritten wurde.

3.3.2.2 hierarchy -check -top

hierarchy -check -top bestimmt die Wurzel des Entwurfs, löst Instanzen von Untermodulen auf, propagiert Parameter und entfernt nicht instanziierte Module. Bei einem einzelnen Modul scheint der Pass wirkungslos, verrichtet jedoch zwei in der Statistik unsichtbare Arbeiten: Er setzt das Attribut `\top` und entfernt `\cells_not_processed`. Ohne das `\top`-Attribut wissen des nachfolgenden Passes nicht, welche Ports zu erhalten sind, denn ein Port des Top-Moduls ist ein physikalischer Pin und kein optimierbares Netz. Die Option `-check` wandelt ein nicht auffindbares Modul in einen harten Fehler um, statt es als Blackbox zu behandeln und eine fehlerhafte Netzliste zu erzeugen.

3.3.2.3 proc

proc ist kein einzelner Pass, sondern eine Folge von zwölf Unterpässen (siehe Tabelle 9: Liste der Unterpässe des Passes "proc"), die sämtlich auch einzeln aufrufbar sind. Sie wirken ausschließlich auf `process`-Objekte. Bei einem Entwurf ohne `always`-Block bleiben alle zwölf Zähler bei null und das RTLIL unverändert.

Tabelle 9: Liste der Unterpasse des Passes "proc"

Unterpasse	Aufgabe
proc_clean	entfernt leere Zweige des Entscheidungsbaums
proc_rmdead	entfernt unerreichbare Zweige, markiert vollständige Verzweigungen
proc_prune	entfernt überschriebene Zuweisungen, wandelt triviale in Verbindungen
proc_init	extrahiert Initialwerte in das Attribut \init
proc_arst	erkennt asynchrone Resets in der Empfindlichkeitsliste
proc_rom	wandelt große konstante Verzweigungen in ROM
proc_mux	wandelt den Entscheidungsbaum in Multiplexer
proc_d latch	erzeugt Latches bei unvollständiger kombinatorischer Logik
proc_dff	erzeugt Flip-Flops aus den Taktsynchronisationen
proc_memwr	wandelt Speicherzugriffe in Zellen
proc_clean	abschließende Bereinigung, entfernt den geleerten Prozess
opt_expr	Konstantenfaltung

Die beiden Unterpasse **proc_mux** und **proc_dff** erzeugen die eigentliche Hardware. Welcher von beiden greift, entscheidet die sync-Zeile des Prozesses:

- **sync always** führt zu rein kombinatorischer Logik,
- **sync posedge** zu Flip-Flops.

3.3.2.4 opt

opt ist ebenfalls ein Sammelbefehl. Er bereinigt die von proc hinterlassenen Alias-Ketten und faltet Konstanten.

Tabelle 10: Liste der Unterpasse des Passes "opt"

Unterpasse	Aufgabe
opt_expr	Konstantenfaltung, Vereinfachung boolescher Ausdrücke
opt_merge	entfernt strukturgleiche Zellen (gemeinsame Teilausdrücke)
opt_muxtree	entfernt unerreichbare Pfade in Multiplexerbäumen
opt_reduce	fasst gleichartige Gatterketten zusammen

opt_dff	vereinfacht Flip-Flops mit konstanten Steuersignalen
opt_clean	entfernt nicht angeschlossene Zellen und Netze

opt_merge ist derjenige Unterpass, der bei arithmetischen Entwürfen tatsächlich Logik einspart, und **opt_clean** beseitigt die Aliase, die `proc` als Nebenprodukt erzeugt.

3.3.2.5 fsm und memory

fsm und memory extrahieren Zustandsautomaten beziehungsweise wandeln Registerfelder in Speicherzellen. Beide wirken nur, wenn der Entwurf entsprechende Konstrukte enthält.

3.3.2.6 techmap

techmap zerlegt die parametrierbaren RTL-Zellen in Ein-Bit-Gatter. Der Wechsel der Schreibweise von `$name` zu `$_NAME_` markiert diesen Ebenenübergang.

3.3.2.7 synth_gatemate

synth_gatemate führt die Technologieabbildung durch und ersetzt sämtliche vorgenannten Schritte einschließlich `hierarchy`. Die Zelltypen verlieren dabei ihr führendes `$`. Solange im Dump `$`-Typen erscheinen, befindet sich der Entwurf noch auf abstrakter Ebene.

3.3.3 Syntheseskripte

Ein häufiges Missverständnis betrifft das Verhältnis der drei möglichen Syntheseskripte zueinander. Sie **ersetzen** einander und ergänzen sich nicht.

3.3.3.1 manuelle Syntheseskripte

die Folge `proc`, `opt`, `fsm`, `opt`, `memory`, `opt`, `techmap` — zielt auf keinen Baustein, obwohl jeder Schritt einzeln beobachtbar ist. Es erzeugt eine generische Netzliste, in der alle Zelltypen mit `$` beginnen. Allerdings ist es für die Fertigung ungeeignet.

3.3.3.2 generische Syntheseskripte

Der Befehl **synth** fasst dieses manuelle Skript zusammen und ergänzt es um weitere Optimierungen. Das Ergebnis bleibt generisch und es entsteht keine einzige herstellerspezifische Zelle.

3.3.3.3 GateMate spezifische Syntheseskripte

Der Befehl **synth_gatemate** ersetzt beide vollständig. Sein erstes Label lädt die Bibliotheken und ruft **hierarchy -check -top** selbst auf. Ein Produktionsskript besteht daher aus dem Einlesen der Quellen, dem Aufruf von **synth_gatemate** und dem Schreiben der Netzliste. Die Zwischenschritte (siehe Tabelle 11: vollständiger Ablauf der Yosys-Syntheseskripte und Abbildung 3: GateMate-spezifischer Syntheseablauf [38]) werden nicht mehr einzeln aufgerufen.

3.3.3.3 Netliste in JSON-Format

Der abschließende Befehl **write_json** exportiert die Netzliste im von nextpnr erwarteten Format. Es handelt sich um denselben Entwurfszustand wie im letzten RTLIL-Dump, lediglich anders serialisiert. Der Export erhält die öffentlichen Namen und die \src-Attribute, sodass sich eine Meldung des Platzierungswerkzeugs bis auf die auslösende Quelltextzeile zurückverfolgen lässt.

Tabelle 11: vollständiger Ablauf der Yosys-Syntheseskripte

Label	Maßgeblicher Befehl	Umformung
begin	read_verilog -lib, hierarchy	lädt die GateMate-Modelle als Blackboxes, bestimmt das Top-Modul
prepare	proc	Entstehung von \$mux, \$dff, \$dlatch aus den Prozessen
prepare	flatten, fsm, wreduce, peepopt, muxpack, share	ebnet die Hierarchie ein, extrahiert Automaten, reduziert überflüssige Wortbreiten, fasst redundante Multiplexer zusammen
prepare	techmap -map cmp2lut.v	Vergleicher werden bereits hier auf LUT mit vier Eingängen abgebildet
map_mult	mul_map.v	\$mul wird zu CC_MULT
coarse	alumacc, memory - nomap	\$add und \$sub verschmelzen zu \$alu und \$macc; Registerfelder werden zu \$mem_v2
map_bram	brams_map.v	\$mem_v2 wird zu CC_BRAM_20K oder CC_BRAM_40K
map_ffram	memory_map	zu kleine Speicher zerfallen in Flip-Flops und Multiplexer

map_gates	techmap, arith_map.v	RTL-Operatoren werden zu Ein-Bit-Gattern; \$alu wird zu CC_ADDF
map_io	iopadmap	Eingang zu CC_IBUF, Ausgang zu CC_OBUF, Tri-State zu CC_TOBUF, bidirektional zu CC_IOBUF
map_regs	dfflegalize, reg_map.v	\$dff und \$dlatch werden auf die vom CPE realisierbaren Varianten normalisiert, dann übersetzt
map_muxs	muxcover, mux_map.v	Multiplexerkaskaden werden zusammengefasst und zu CC_MX2/CC_MX4
map_luts	abc -genlib, lut_tree_map.v, gatmate_foldinv	die verbleibende Logik wird zu CC_LUT2, CC_L2T4, CC_L2T5; Inverter werden in deren Konfiguration absorbiert
map_cells	lut_map.v	letzte Übersetzung in die endgültigen Zellen
map_bufg	clkbufmap	Takte werden über CC_BUFG geführt
check	stat-width, check-noinit	Abschlusszählung und Konsistenzprüfung

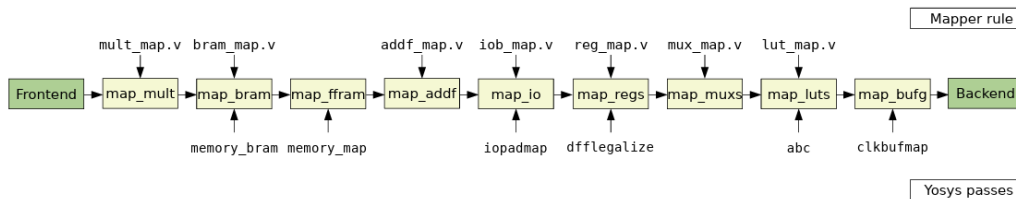


Abbildung 3: GateMate-spezifischer Syntheseablauf [38]

Ohne die Option **-luttree** tritt an die Stelle der LUT-Baum-Abbildung ein klassisches Mapping auf LUT mit vier Eingängen, das für das ältere Werkzeug **PnR** vorgesehen ist. Die Optionen **-nobram**, **-noaddf**, **-nomult** und **-nomx4** überspringen jeweils genau eine Zeile dieser Tabelle und zwingen die betreffende Logik in LUTs. (siehe Appendix B.2 Synthese mit Yosys (synth_gatmate))

Von fünfzehn erfassten Zuständen unterscheiden sich lediglich drei vom jeweiligen Vorgänger (siehe Tabelle 12: gemessener Ablauf der Yosys-Syntheseskripte). Dies ist kein Mangel des Ablaufs, sondern Ausdruck seiner Allgemeingültigkeit, denn derselbe Ablauf muss Speicher, Automaten, Multiplizierer und Register gleichermaßen behandeln können.

Tabelle 12: gemessener Ablauf der Yosys-Syntheseskripte

Label	Zellen im Modul	
read_verilog	1 × \$and	
begin bis map_ffram	1 × \$and	unverändert
map_gates	1 × \$_AND_	Änderung
map_io	1 × \$_AND_, 2 × CC_IBUF, 1 × CC_OBUF	Änderung
map_regs, map_muxs	unverändert	
map_luts	1 × CC_LUT2, 2 × CC_IBUF, 1 × CC_OBUF	Änderung
map_cells bis check	unverändert	

3.4 nextpnr-himbaechel

nextpnr-himbaechel ist das für GateMate optimierte Backend für Platzierung und Verdrahtung. Es übernimmt die JSON-Netzliste von Yosys und erzeugt die Implementierungsdatei für gmpack. (siehe Abbildung 2: Detaillierter Workflow von einem Verilog-Code bis zum Silizium unter Verwendung von Yosys für die Synthese und NextPnr für das Routing und die Platzierung)

Die Arbeitsteilung ist eindeutig: **Yosys ermittelt, welche Primitive verwendet werden, während nextpnr deren Position in der Matrix und die Art und Weise ihrer Verbindung untereinander bestimmt** (siehe Abbildung 4: Überblick des nextpnr-Tools [38]). Die erreichbare Taktfrequenz ergibt sich erst aus dem Zusammenwirken beider Schritte.

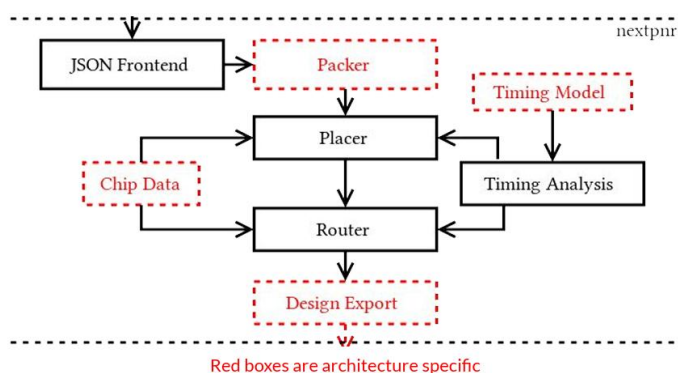


Abbildung 4: Überblick des nextpnr-Tools [38]

3.4.1 Packing

Zusammengehörige Logik- und Registerprimitiven, etwa zwei CC_L2T4 und zwei CC_DFF werden beim Packing in dieselben physikalischen CPEs gebündelt. Dies ist der Schritt, in dem sich die Zellzahl aus in eine CPE-Zahl übersetzt.

3.4.2 Placement

Da werden gepackten CPEs mit Timing-getrieben auf konkrete Positionen des GateMate-Gitters zugewiesen. Der Algorithmus wird über **--placer** gewählt, die Pin-Zuweisung über eine CCF-Datei vorgegeben.

3.4.3 Routing

Das Routing ist die Verbindung der platzierten Elemente über die Routing-Struktur. Für GateMate ist **--router** router2 vorgesehen, um die Verwendung der für diese Architektur geeigneten Routing-Engine zu erzwingen. Optionale Zeitvorgaben werden über eine SDC-Datei eingelesen, die Ergebnisse über **--report** als JSON ausgegeben. (siehe Appendix B.3 Implementierung: Place & Route mit nextpnr-himbaechel)

Aufruf:

```
nextpnr-himbaechel \  
  --device CCGM1A1 \  
  --json <synthese_rtlil_file>.json \  
  -o ccf=<constraint_file>.ccf \  
  -o out=impl.txt \  
  --router router2 \  
  --report report.json
```

Anschließend erzeugt **gmpack** impl.txt <entwurf>.bit den Bitstrom, der mit **openFPGALoader** über JTAG oder SPI geladen wird.

4 Praktische Umsetzung: Volladdierer

4.1 Zielsetzung und Versuchsaufbau

Dieses Kapitel überprüft die in Kapitel 3 **Theorie: RTLIL, Yosys und nextpnr** dargestellte Theorie anhand eines minimalen, aber vollständigen Entwurfs. Der Volladdierer eignet sich hierfür, weil er alle Grundoperationen enthält, ohne Register oder Speicher zu erfordern, und weil seine Funktion vollständig von Hand nachrechenbar ist. Untersucht werden **drei funktional identische Beschreibungen** desselben Bauteils. Sämtliche Quelldateien, Syntheseskripte und Zwischenergebnisse dieses Kapitels sind unter [38] abrufbar.

4.1.1 Variante A — Datenflussbeschreibung

Die Funktion wird als boolescher Ausdruck mit kontinuierlichen Zuweisungen formuliert (siehe Appendix C.1 Voll_addierer ohne Process):

```
assign sum = a ^ b ^ cin;  
assign cout = (a & b) | (a & cin) | (b & cin);
```

4.1.2 Variante B — Prozessbeschreibung

Dieselbe Funktion wird als vollständige Wahrheitstabelle in einem **always**-Block formuliert. (siehe Appendix C.2 Voll_addierer mit Process):

```
always @(*) begin  
    sum = a ^ b ^ cin;  
    co  = (a & b) | (a & cin) | (b & cin);  
end
```

4.1.3 Variante C — Prozessbeschreibung mit case-Verzweigung

Dieselbe Funktion wird als vollständige Wahrheitstabelle in einem **always**-Block mit **case**-Verzweigung formuliert, mit je einem Zweig für die acht Eingangskombinationen. (siehe Appendix C.3 Voll_addierer mit Process und case)

```
always @(*) begin  
    case ({a, b, cin})  
        3'b000: begin sum = 1'b0; cout = 1'b0; end  
    endcase  
end
```

```

3'b001: begin sum = 1'b1; cout = 1'b0; end
3'b010: begin sum = 1'b1; cout = 1'b0; end
3'b011: begin sum = 1'b0; cout = 1'b1; end
3'b100: begin sum = 1'b1; cout = 1'b0; end
3'b101: begin sum = 1'b0; cout = 1'b1; end
3'b110: begin sum = 1'b0; cout = 1'b1; end
3'b111: begin sum = 1'b1; cout = 1'b1; end

endcase

end

```

Die drei Varianten wurden mit identischen Optionen synthetisiert (`-luttree -nomx8`) und der RTLIL-Zustand nach jedem der vierzehn Labels gesichert. Die Hypothese lautete, dass sich die Beschreibungen nach spätestens zwei Schritten angleichen, da sie dieselbe boolesche Funktion darstellen. (siehe Tabelle 13: gemessener Verlauf der Synthese eines Volladdierers in drei Beschreibung variante)

Tabelle 13: gemessener Verlauf der Synthese eines Volladdierers in drei Beschreibung variante

Label	Variante A (assign)	Variante B (always)	Variante C (always/case)
read_verilog	3 × \$and, 1 × \$or, 2 × \$xor, 2 Inverter	1 process, 7 cells	1 process, 0 cells
hierarchy	unverändert	unverändert	unverändert
begin	unverändert	unverändert	unverändert
prepare	3 × \$and, 2 × \$or, 2 × \$xor, kein Inverter	2 × \$and, 1 × \$or, 2 × \$xor, Kein Prozess	1 × \$meminit, 1 × \$memrd_v2
map_mult	unverändert	unverändert	unverändert

coarse	unverändert	unverändert	1 × \$mem_v2
map_bram	unverändert	unverändert	unverändert
map_ffram	unverändert	unverändert	5 × \$mux
map_gates	3 × \$ _AND_, 1 × \$ _OR_, 2 × \$ _XOR_	2 × \$ _AND_, 1 × \$ _OR_, 2 × \$ _XOR_	6 × \$ _MUX_, 1 × \$ _NOT_
map_io	+ 3 × CC_IBUF, 2 × CC_OBUF	+ 3 × CC_IBUF, 2 × CC_OBUF	+ 3 × CC_IBUF, 2 × CC_OBUF
map_regs	unverändert	unverändert	unverändert
map_muxs	unverändert	unverändert	2 × CC_MX4, 1 × \$ _NOT_
map_luts	3 × CC_L2T4	3 × CC_L2T4	2 × CC_MX4, +1 × CC_LUT1
map_cells ... check	unverändert	unverändert	unverändert

Dieselbe Funktion führt zu **zwei vollständig verschiedenen physikalischen Realisierungen**. (siehe Tabelle 14: Vergleich der physikalischen Zellen der Varianten des Volladdierers nach der Gate-Synthese)

Tabelle 14: Vergleich der physikalischen Zellen der Varianten des Volladdierers nach der Gate-Synthese

	Variante A & B	Variante C
Logikzellen	3 × CC_L2T4	2 × CC_MX4, 1 × CC_LUT1
E/A-Zellen	3 × CC_IBUF, 2 × CC_OBUF	identisch

4.2 Analyse des Datenflusspfades (Variante A)

Der Verlauf entspricht der Erwartung und drei Labels verändern den Entwurf. Bei **read_verilog** werden die Zuweisungen direkt in zellen umgesetzt, da es sich um kombinatorische Ausdrücke handelt. (siehe Abbildung 5: Netzliste nach dem Einlesen, Variante A. Die kontinuierlichen Zuweisungen wurden unmittelbar in sechs RTL-Operatoren überführt.)

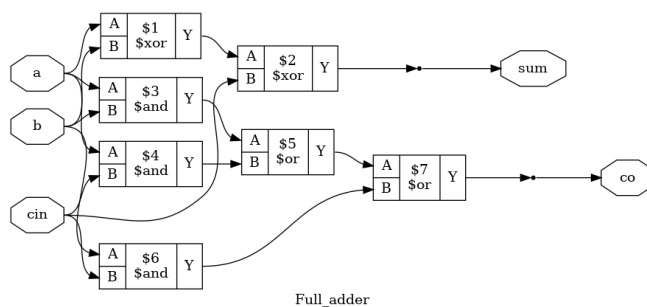


Abbildung 5: Netzliste nach dem Einlesen, Variante A. Die kontinuierlichen Zuweisungen wurden unmittelbar in sechs RTL-Operatoren überführt.

Bei **prepare** entfällt eine der drei XOR-Zellen. Der Term $a \wedge b$ tritt im Quelltext zweimal auf in sum und in cout. Die \src-Attribute belegen die Herkunft aus Zeile 5 beziehungsweise Zeile 6. Der Unterpass opt_merge erkennt die strukturelle Gleichheit und entfernt eine Zelle. Dies ist die einzige Stelle des gesamten Ablaufs, an der tatsächliche Logik eingespart wird. (siehe Abbildung 6: Netzliste nach prepare. Von den drei XOR-Zellen ist eine entfallen.)

Kommentiert [YS1]: Muss noch umformuliert

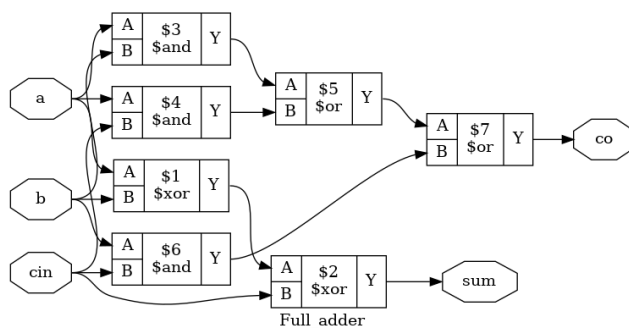


Abbildung 6: Netzliste nach prepare. Von den drei XOR-Zellen ist eine entfallen.

Bei **map_gates** werden die parametrierbaren RTL-Operatoren in Ein-Bit-Gatter zerlegt.. Der Ebenenwechsel ist an den Unterstrichen der Zelltypen ablesbar: Aus \$xor wird \$_XOR_, aus fünf Parametern wird keiner. (siehe Abbildung 7: Übergang auf die Gatterebene. Die parametrierbaren Operatoren sind durch Ein-Bit-Gatter ersetzt.)

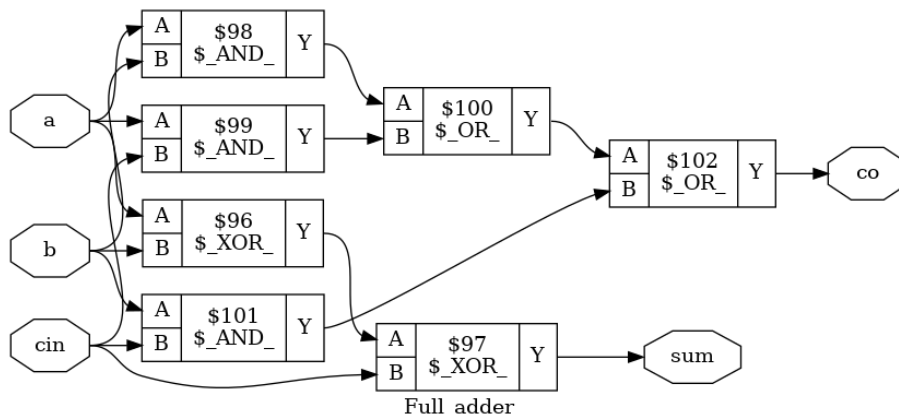


Abbildung 7: Übergang auf die Gatterebene. Die parametrierbaren Operatoren sind durch Ein-Bit-Gatter ersetzt.

Bei **map_io** werden die Anschlüsse vor der Logik auf CC_-Primitive abgebildet, da sie physikalische Randbedingungen darstellen. Das Attribut \keep schützt die Tampons vor nachfolgenden Optimierungen. (siehe Abbildung 8: Einfügen der Pad-Treiber. Die Ports erhalten ihre physikalischen Primitive, bevor die Logik abgebildet wird.)

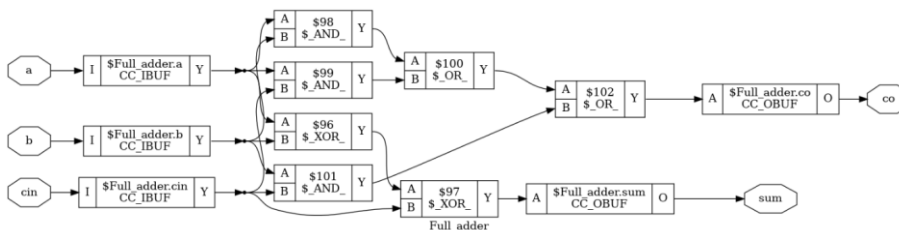


Abbildung 8: Einfügen der Pad-Treiber. Die Ports erhalten ihre physikalischen Primitive, bevor die Logik abgebildet wird.

Bei **map_luts** verschwinden die fünf Gatter vollständig. An ihre Stelle treten drei CC_L2T4, deren Konfigurationsparameter sich zurücklesen lassen (siehe Abbildung 9: Endgültige Netzliste, Variante A. Die fünf Gatter sind in drei LUT-Bäumen aufgegangen.):

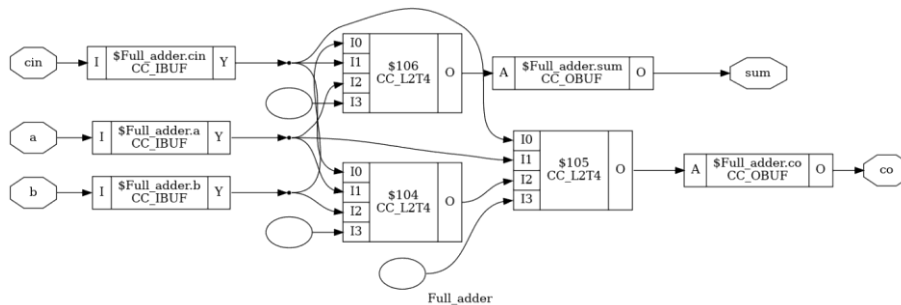


Abbildung 9: Endgültige Netzliste, Variante A. Die fünf Gatter sind in drei LUT-Bäumen aufgegangen.

Die Rekonstruktion stimmt exakt mit dem Quelltext überein. Der Appendix D.1 Simulation der Synthesenergebnisse der Variante A/B mit logic.ly zeigt die Simulation des Volladdierers in Abhängigkeit von der Konfiguration der LUTs der Tabelle 15: Konfiguration des LUTs für den Volladdierer. Damit ist der Weg von der Beschreibung bis zur Konfigurationsbitfolge lückenlos nachvollzogen und die Aussage aus Abschnitt 3.2.5.2 Look-Up-Tables belegt: In der Logikmatrix existieren keine Gatter mehr, sondern ausschließlich Speicherinhalte.

Tabelle 15: Konfiguration des LUTs für den Volladdierer für die Variante A und B

Zelle	INIT_L00 (\$ 104)	INIT_L01 (\$ 106)	INIT_L10 (\$ 105)	Funktion
1	4'0110 (XOR)	4'1010 (Durchleitung)	4'1000 (AND)	$(a \wedge b) \& cin$
2	4'0110 (XOR)	4'1010 (Durchleitung)	4'0110 (XOR)	$a \wedge b \wedge cin \rightarrow sum$
3	4'1000 (AND)	4'1010 (Durchleitung)	4'1110 (OR)	$(a \& b) \mid ((a \wedge b) \& cin) \rightarrow cout$

4.3 Analyse des Prozesspfades (Variante B)

Der Ablauf unterscheidet sich nur bei „read_verilog“ insofern, als der „always“-Block nicht in Zellen umgewandelt, sondern in Form eines RTLIL::Process gespeichert wird; nach dem Einlesen mit „read_verilog“ verschwindet er jedoch vollständig, und die übrigen Schritte entsprechen denen der Variante A. (siehe Abbildung 10: Netzliste nach dem Einlesen, Variante B.)

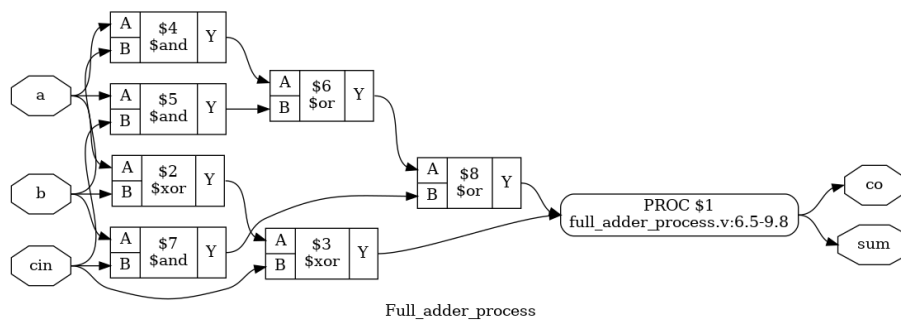
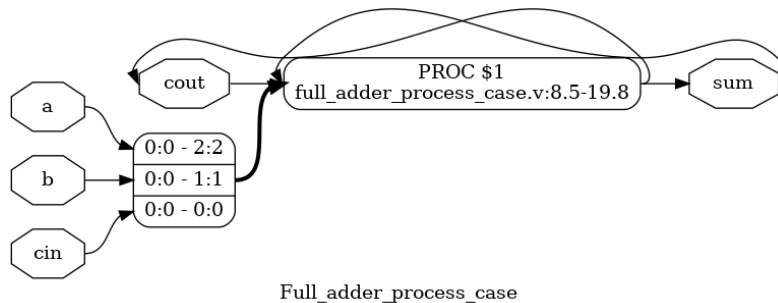


Abbildung 10: Netzliste nach dem Einlesen, Variante B.

4.4 Analyse des Prozess- und Case-Pfades (Variante C)

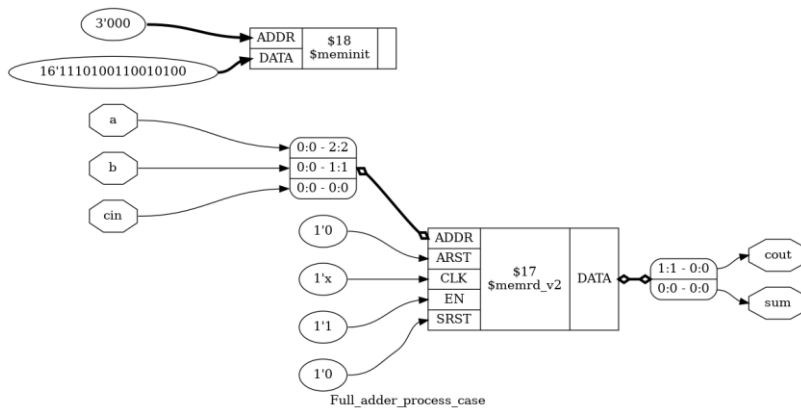
Hier zeigt sich der grundlegende Unterschied. Der always-Block wird nicht in Zellen überführt, sondern als RTLIL::Process abgelegt (siehe Abbildung 11: Zustand nach dem Einlesen, Variante B. Es existiert keine Zelle; der gesamte Entwurf liegt als Prozess vor.). Das Schaltbild ist entsprechend leer, da show Prozesse keinen für sich aussagekräftiger Befund darstellen kann. Der Appendix D.2 Simulation der Synthesenergebnisse der Variante C mit simulator.io [9] zeigt die Simulation des Volladdierers in Abhängigkeit von der Konfiguration der LUTs der Tabelle 16: Konfiguration des LUTs für den Volladdierer für die Variante C mit Process und case. (siehe Abbildung 17: Konfiguration der Zelle aus der Synthese Ergebnisse der Variante C [38])



Full_adder_process_case

Abbildung 11: Zustand nach dem Einlesen, Variante B. Es existiert keine Zelle; der gesamte Entwurf liegt als Prozess vor.

Der Unterpass **proc_rom** meldet im Protokoll Converted 1 switch. Da die acht Zweige ausschließlich Konstanten zuweisen, interpretiert Yosys die Verzweigung nicht als Logik, sondern als Tabelle. Es entstehen **\$meminit** und **\$memrd_v2** statt Multiplexern. (siehe Abbildung 12: Nach prepare. Die case-Verzweigung wurde als Nachschlagetabelle erkannt.)



Full_adder_process_case

Abbildung 12: Nach prepare. Die case-Verzweigung wurde als Nachschlagetabelle erkannt.

Der Pass **memory** fasst die Teilzellen zu einer **\$mem_v2** zusammen mit den Parametern **ABITS = 3** und **INIT = 16'1110100110010100**. Die vollständige Wahrheitstabelle des Volladdierers ist hier als geschlossener Speicherinhalt sichtbar: drei Adressbits für acht Kombinationen, sechzehn Bit für die zwei Ausgänge je Kombination. (siehe Abbildung 13: Nach coarse. Die Tabelle liegt als einzelne Speicherzelle vor.)

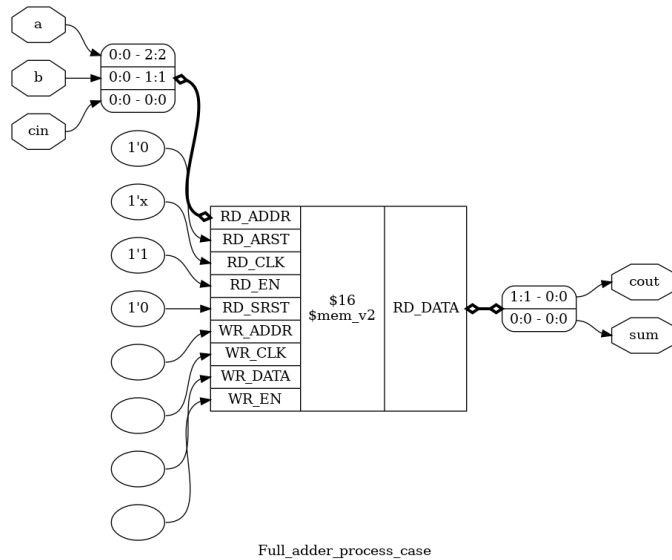


Abbildung 13: Nach coarse. Die Tabelle liegt als einzelne Speicherzelle vor.

Bei **map_bram** wurde geprüft, ob sich der Speicher in einem Block-RAM unterbringen lässt; mit sechzehn Bit ist er dafür zu klein. **memory_map** zerlegt ihn daher wieder in fünf **\$mux**-Zellen. (siehe Abbildung 14: Nach map_ffram. Die Speicherzelle ist in Multiplexer zerfallen.)

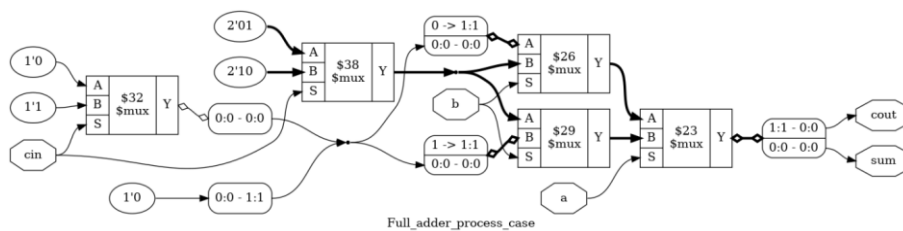


Abbildung 14: Nach map_ffram. Die Speicherzelle ist in Multiplexer zerfallen.

muxcover erkennt die Kaskadenstruktur und bildet sie auf die CC_MX4-Primitive ab, die das CPE ohne Belegung einer Wahrheitstabelle realisiert. (siehe Abbildung 15: Nach map_muxs. Sechs Ein-Bit-Multiplexer sind zu zwei nativen Vierfach-Multiplexern zusammengefasst.)

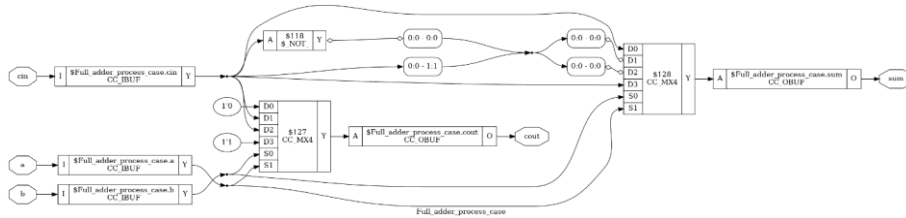


Abbildung 15: Nach map_muxs. Sechs Ein-Bit-Multiplexer sind zu zwei nativen Vierfach-Multiplexern zusammengefasst.

Es verbleibt ein einzelner Inverter, der zu einem CC_LUT1 mit $INIT = 2'01$ wird. Die Beschaltung der Multiplexer ist in Tabelle 16: Konfiguration des LUTs für den Volladdierer für die Variante C mit Process und case ausgewertet. (siehe Abbildung 16: Endgültige Netzliste, Variante B. Zwei Vierfach-Multiplexer und ein Inverter.)

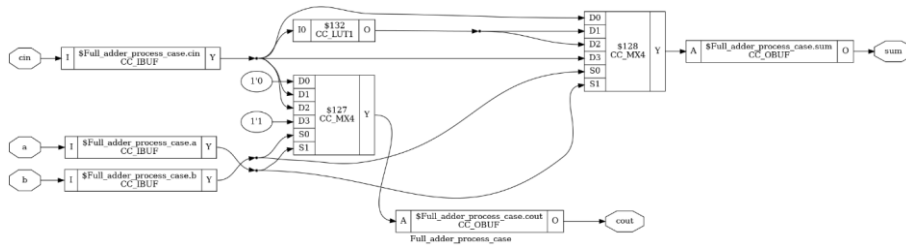


Abbildung 16: Endgültige Netzliste, Variante B. Zwei Vierfach-Multiplexer und ein Inverter.

Tabelle 16: Konfiguration des LUTs für den Volladdierer für die Variante C mit Process und case

N°	Type	D0	D1	D2	D3	S1	S0	output
\$132	LUT	cin	-	-	-	-	-	ncin
\$128	MUX4	cin	ncin	ncin	cin	a	b	$a \wedge b \wedge cin \rightarrow \text{sum}$
\$127	MUX4	0	cin	cin	1	a	b	$(a \& b) \mid ((a \wedge b) \& cin) \rightarrow \text{cout}$

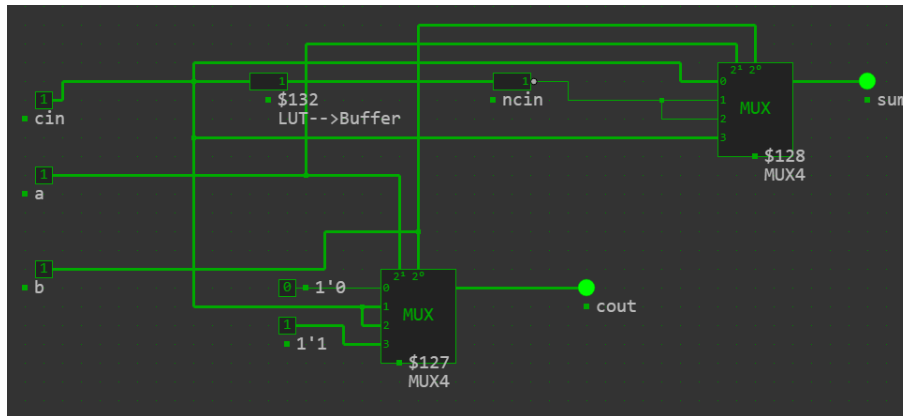


Abbildung 17: Konfiguration der Zelle aus der Synthese Ergebnisse der Variante C [38]

4.5 Place & Route

Die von Yosys erzeugte JSON-Netzliste wird an nextpnr-himbaechel übergeben. Da beide Varianten unterschiedliche Primitive enthalten, unterscheidet sich bereits der Ausgangspunkt des Packings. Die **4.1.1 Variante A — Datenflussbeschreibung** und **4.1.2 Variante B — Prozessbeschreibung** liefern drei LUT-Bäume, und die **4.1.3 Variante C — Prozessbeschreibung mit case** zwei Multiplexer und eine LUT.

Von besonderem Interesse ist, ob sich der in Abschnitt **4.6 Bewertung** festgestellte Vorteil der Prozessvariante nach dem Packing bestätigt: Zwei CC_MX4 und eine CC_LUT1 könnten gegebenenfalls in weniger CPEs gebündelt werden als drei CC_L2T4, da ein CPE zwei unabhängige Vierfach-Funktionen aufnehmen kann. (siehe Tabelle 17: Ressourcenbelegung nach dem Packing)

Tabelle 17: Ressourcenbelegung nach dem Packing

Ressource	Verfügbar	Variante A/B	Variante C (process_case)
CPE_LT	40 960	3	3
CPE_CPLINES	20 480	3	0
CPE_MX4	—	0	2

CPE_BRIDGE	—	1	1
CPE_COMP	20 480	0	0
CPE_FF	40 960	0	0
CPE_RAMIO	40 960	2	2
GPIO	162	5	5
IOSEL	162	5	5
RAM_HALF	64	0	0
PLL	4	0	0

4.6 Bewertung der Ergebnisse

Diese Messung widerlegt die Annahme, dass der Schreibstil keinen Einfluss auf das Ergebnis der Synthese habe, zeigt aber gleichzeitig, dass sich dieser Unterschied nicht zwangsläufig in einem Unterschied bei den Ressourcen niederschlägt. Dabei sind drei Punkte zu beachten.

4.6.1 Yosys bildet die geschriebene Struktur ab, nicht die boolesche Bedeutung

Eine Verhaltensbeschreibung mit konstanten Verzweigungen wird als Array interpretiert, während ein boolescher Ausdruck als Logik interpretiert wird. Beide Interpretationen sind korrekt, führen jedoch zu unterschiedlichen Primitiven: drei CC_L2T4 gegenüber zwei CC_MX4 und einem CC_LUT1.

4.6.2 Die Netzliste ist keine Ressourcenmessung

Es könnte vermutet werden, dass diese verschiedenen Primitive unterschiedliche Ressourcenanforderungen mit sich bringen. Die Berichte von nextpnr widerlegen diese Annahme, da nach dem Packing beide Varianten jeweils drei CPE_LT-Elemente belegen.

Der Unterschied betrifft Teilressourcen innerhalb des CPE. Die Varianten A und B verwenden drei schnelle Verbindungspfade (CPE_CPLINES), während die Variante C zwei native Multiplexer (CPE_MX4) nutzt.

Die CPE-Architektur unterstützt somit alle drei Ausführungsvarianten mit demselben Elementverbrauch. Dies ist kein Zufall, da mit ein und demselben Element wahlweise Lookup-Tabellenbäume (LUTs), Multiplexer oder Addierer realisiert werden können, sodass verschiedene Zuordnungswege zur selben physikalischen Ressource führen. Bei Architekturen mit getrennten Ressourcenquoten wäre eine solche Angleichung nicht zu erwarten.

Aus methodischer Sicht folgt daraus, dass eine Bewertung des Beschreibungsstils, die sich ausschließlich auf die Zählung der Zellen nach der Synthese stützt, zu einer falschen Schlussfolgerung führt.

4.6.3 Die Erkennung als ROM skaliert nicht beliebig

Bei umfangreicheren Tabellen würde map_bram eingreifen und das Projekt blockweise in den RAM-Speicher verschieben – mit einem völlig anderen zeitlichen Verhalten und einer anderen Ressourcenklasse. Der Umschaltzeitpunkt liegt dort, wo der Inhalt der Tabelle die Mindestgröße eines CC_BRAM_20K erreicht. Die hier beobachtete Gleichwertigkeit der Varianten hängt also mit der geringen Größe des Beispiels zusammen und lässt sich nicht verallgemeinern.

In der Praxis bedeutet dies, dass der Ressourcenbericht nach dem „Place & Route“ überprüft werden muss und weder aus der Beschreibung noch aus der Verbindungsliste abgeleitet werden darf.

Fazit

Diese Arbeit hat Schritt für Schritt den Weg einer Hardwarebeschreibung vom Quellcode bis zur Konfiguration des GateMate-FPGAs nachgezeichnet. Dabei stützte sie sich auf die Aufteilung des Synthesevorgangs in vierzehn einzelne Schritte und auf die Speicherung des internen Zustands nach jedem Schritt.

Yosys speichert das Design von Anfang bis Ende in Form einer einzigen Struktur im Arbeitsspeicher, dem RTLIL; jeder Durchlauf modifiziert diese in situ. Die Front-End-Analyse bestätigte, dass die ersten Synthese-Schritte bereits beim Einlesen stattfinden, d. h. die Operationsknoten des AST werden sofort in Zellen umgewandelt, während die Knoten, die das Verhalten beschreiben, über einen Prozessgenerator umgewandelt werden. Von den vierzehn Parametern des Befehls „synth_gatemate“ verändern je nach Beschreibungsstil drei bis sechs das Design des untersuchten Volladdierers; die wichtigsten sind „prepare“, „coarse“, „map_gates“ und „map_luts“.

Ein Boolescher Ausdruck und eine vollständige Wahrheitstabelle in einem „case“-Block führten zu völlig unterschiedlichen Verknüpfungslisten. Die Beschreibung des Datenflusses erzeugte drei CC_L2T4-LUT-Bäume, und die Prozessbeschreibung wurde von `proc_rom` als Nachschlagetabelle erkannt und schließlich auf zwei CC_MX4 und eine CC_LUT1 abgebildet. Yosys bildet somit die schriftliche Struktur ab, nicht die boolesche Bedeutung. Beide Implementierungen konnten vollständig nachverfolgt werden, was zudem belegt, dass die Logikmatrix keine Logikgatter enthält, sondern ausschließlich Konfigurationen und Signalfade.

Nach dem von nextpnr durchgeführten „Packing“ belegen die beiden Varianten jedoch jeweils drei CPE_LT-Elemente. Der Unterschied zwischen diesen Varianten betrifft Teilressourcen innerhalb des CPE. Daraus folgt, dass die Zählung der Zellen nach der Synthese kein Maß für die Ressourcen darstellt, sondern nur der Bericht des Platzierungstools zuverlässige Zahlen liefert.

Eine zeitliche Analyse war aufgrund fehlender Register nicht möglich, und die Bezeichnungen für Speicher, Multiplikatoren und Register blieben ohne Wirkung. Eine sequenzielle Variante würde es erstmals ermöglichen, eine Frequenz anzugeben, während eine dritte Beschreibung mit verschachtelten „if“-Anweisungen einen weiteren Darstellungsweg aufzeigen würde. Die offene Struktur des RTLIL ermöglicht es zudem, eigene Durchläufe zu entwickeln, wodurch man den Ablauf nicht nur beobachten, sondern auch gezielt erweitern könnte.

Litteraturverzeichnis

Normen, Datenblätter und Herstellerdokumentation (Cologne Chip AG)

- [1] **Cologne Chip AG (2025):** *UG1001 — GateMate FPGA Primitives Library User Guide*. Version März 2025, 129 S.
- [2] **Cologne Chip AG (2026):** *UG1002 — GateMate FPGA Toolchain Installation User Guide*. Version Mai 2026.
- [3] **Cologne Chip AG:** *GateMate FPGA — Produktseite und aktuelle Builds*. Verfügbar unter: <https://www.colognechip.com/programmable-logic/gatemate> (Abgerufen im Rahmen des Projekts).
- [4] **Cologne Chip AG:** *GateMate FPGA Toolchain Quickstart*. Online-Schnellstartanleitung. Verfügbar unter: <https://colognechip.com/programmable-logic/gatemate/gatemate-toolchain-quickstart/>
- [5] **Cologne Chip AG:** *Datenblätter DS1001, DS1002, DS1003* (Referenziert in UG1002).

Wissenschaftliche Arbeiten und Literatur

- [1] **Parnow, C. (2022):** *Analyse der Yosys Prozesse, die Verilog in die interne Datenstruktur RTLIL konvertieren*. Bachelorarbeit, TH Köln und Fachhochschule Dortmund. Zweitprüfer: Prof. Dr.-Ing. M. Karagounis.
- [2] **Koers, L.:** *Masterthesis*. Fachhochschule Dortmund. Verfügbar unter OPUS Baden-Württemberg: https://opus.bsz-bw.de/fhdo/frontdoor/deliver/index/docId/3803/file/MasterThesis_LarsKoers.pdf.

Software-Dokumentation, Frameworks und Open-Source-Distributionen

- [1] **YosysHQ:** *Yosys Open Synthesis Suite — Handbuch und Dokumentation der PASSES*. Verfügbar unter: <https://yosyshq.net/yosys>.
- [2] **FHDO-ICLAB:** *Yosys Fork — Beispiele für Python-APIs*. GitHub-Repository: <https://github.com/FHDO-ICLAB/yosys>.
- [3] **YosysHQ:** *OSS CAD Suite — Precompiled Packages*. GitHub-Repository: <https://github.com/YosysHQ/oss-cad-suite-build/releases>.
- [4] **Lokale Entwicklungsumgebung:** *Yosys-Hilfsprogramme (help <command>) und Verzeichnis share/yosys/gatemate/* (Lokale Installationsbasis).

- [5] **YosysHQ:** Yosys Open Synthesis Suite — The RTL Intermediate Language (RTLIL).
verfügbar unter:
https://yosyshq.readthedocs.io/projects/yosys/en/v0.52/yosys_internals/formats/rtlil_rep.html
- [6] **Partick Urban, Cologne Chip AG:** GateMate Synthesis Keynote FPGA Conference 2021. Verfügbar unter:
<https://colognechip.com/wp-content/uploads/GateMate-Synthesis-Keynote-FPGA-Conf.2021.pdf>
- [7] **FPGA Ignite nextpnr:** Open Source FPGA place & route. Verfügbar unter:
https://fpga-ignite.github.io/fpga_ignite_2023/nextpnr/FPGA_Ignite_nextpnr.pdf
- [8] GitHub Repository: Masterstudienarbeit, yvsim001. Verfügbar unter:
<https://github.com/yvsim001/Masterstudienarbeit>
- [9] Simulation Link der Variante B. Verfügbar unter:
<https://simulator.io/board/TGZppuCKs/1>

Online- und Videoquellen

- [1] **YouTube:** Yosys Installation on Ubuntu 22.04.5 LTS. Videoanleitung.
<https://www.youtube.com/watch?v=dhdPI0LOzo8>.
- [2] **YouTube:** Ergänzende Videoressource.
https://www.youtube.com/watch?v=AMkFuS_ZkrM.

Abkürzungsverzeichnis

- ABC** — Logic Synthesis and Verification System (Berkeley)
- AIG** — And-Inverter Graph
- ALU** — Arithmetic Logic Unit
- ASIC** — Application-Specific Integrated Circuit
- AST** — Abstract Syntax Tree
- BLIF** — Berkeley Logic Interchange Format
- BRAM** — Block RAM
- CCF** — Cologne Constraint File

CPE — Cologne Programmable Element
DPSRAM — Dual-Ported Static RAM
DSP — Digital Signal Processing
EDIF — Electronic Design Interchange Format
FF — Flip-Flop
FFT — Fast Fourier Transform
FIR — Finite Impulse Response
FPGA — Field-Programmable Gate Array
FSM — Finite State Machine
GMPACK — der für Cologne Chip bestimmte Bitstream-Generator.
GPIO — General-Purpose Input/Output
HDL — Hardware Description Language
INIT — Initialisation Vector / Truth Table Parameter
JTAG — Joint Test Action Group (Test- und Programmierschnittstelle)
LSB — Least Significant Bit
LUT — Look-Up Table
MAC — Multiply-Accumulate
MSB — Most Significant Bit
MUX — Multiplexer
PLL — Phase-Locked Loop
PnR — Place and Route
RTL — Register-Transfer Level
RTLIL — Register-Transfer Level Intermediate Language
SDC — Synopsys Design Constraints
SerDes — Serializer-Deserializer
SPI — Serial Peripheral Interface
SRAM — Static Random-Access Memory
VHDL — VHSIC Hardware Description Language

Abbildungverzeichnis

Abbildung 1: CPE-Logikelement[2]	8
Abbildung 2: Detaillierter Workflow von einem Verilog-Code bis zum Silizium unter Verwendung von Yosys für die Synthese und NextPnr für das Routing und die Platzierung	10
Abbildung 3: GateMate-spezifischer Syntheseablauf [38]	20
Abbildung 4: Überblick des nextpnr-Tools [38]	21
Abbildung 5: Netzliste nach dem Einlesen, Variante A. Die kontinuierlichen Zuweisungen wurden unmittelbar in sechs RTL-Operatoren überführt.	26
Abbildung 6: Netzliste nach prepare. Von den drei XOR-Zellen ist eine entfallen.....	26
Abbildung 7: Übergang auf die Gatterebene. Die parametrierbaren Operatoren sind durch Ein-Bit-Gatter ersetzt	27
Abbildung 8: Einfügen der Pad-Treiber. Die Ports erhalten ihre physikalischen Primitive, bevor die Logik abgebildet wird.....	27
Abbildung 9: Endgültige Netzliste, Variante A. Die fünf Gatter sind in drei LUT-Bäumen aufgegangen.....	28
Abbildung 10: Netzliste nach dem Einlesen, Variante B.	29
Abbildung 11: Zustand nach dem Einlesen, Variante B. Es existiert keine Zelle; der gesamte Entwurf liegt als Prozess vor.	30
Abbildung 12: Nach prepare. Die case-Verzweigung wurde als Nachschlagetabelle erkannt.....	30
Abbildung 13: Nach coarse. Die Tabelle liegt als einzelne Speicherzelle vor.	31
Abbildung 14: Nach map_ffram. Die Speicherzelle ist in Multiplexer zerfallen.	31
Abbildung 15: Nach map_muxs. Sechs Ein-Bit-Multiplexer sind zu zwei nativen Vierfach-Multiplexern zusammengefasst.	32
Abbildung 16: Endgültige Netzliste, Variante B. Zwei Vierfach-Multiplexer und ein Inverter.....	32
Abbildung 17: Konfiguration der Zelle aus der Synthese Ergebnisse der Variante C [38]	33
Abbildung 18: Konfiguration der LUTs für die Funktion $(a \wedge b) \& cin$	56
Abbildung 19: Konfiguration der LUTs für die Funktion $a \wedge b \wedge c$	56
Abbildung 20: Konfiguration der LUTs für die Funktion $(a \& b) \mid ((a \wedge b) \& cin) \rightarrow cout$	56
Abbildung 21: Simulation der Addition $a(1) + b(0)$, $cin(0)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B.....	57
Abbildung 22: Simulation der Addition $a(0) + b(1)$, $cin(0)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B.....	58
Abbildung 23: Simulation der Addition $a(1) + b(1)$, $cin(0)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B.....	59

Abbildung 24: Simulation der Addition $a(0) + b(1)$, cin (1) mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B.....	60
Abbildung 25: Simulation der Addition $a(1) + b(1)$, cin (1) mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B.....	61
Abbildung 26: Simulation der Addition $a(1) + b(0)$, cin (0) mit der von Yosys konfigurierter LUT-Tabelle der Variante C.....	62
Abbildung 27: Simulation der Addition $a(0) + b(1)$, cin (0) mit der von Yosys konfigurierter LUT-Tabelle der Variante C.....	62
Abbildung 28: Simulation der Addition $a(1) + b(1)$, cin (0) mit der von Yosys konfigurierter LUT-Tabelle der Variante C.....	62
Abbildung 29: Simulation der Addition $a(1) + b(0)$, cin (1) mit der von Yosys konfigurierter LUT-Tabelle der Variante C.....	63
Abbildung 30: Simulation der Addition $a(1) + b(1)$, cin (1) mit der von Yosys konfigurierter LUT-Tabelle der Variante C.....	63

Tabellenverzeichnis

Tabelle 1: Ressourcen des CCGM1A1 (Evaluation Board)	7
Tabelle 2: Abstraktionsebenen der FPGA-Entwurf	8
Tabelle 3: Erkennungsmerkmal im Dump.....	9
Tabelle 4: Beispiel für die Zuordnung zwischen einem AST-Knotentyp und einem RTLIL- Objekt	11
Tabelle 5: Zuweisung eines bestimmten Befehls in Textform in der Klasse RTLIL C++.....	12
Tabelle 6: Syntheseergebnisse zwei verschiedenen Notationen.....	13
Tabelle 7: Beispiel Konfigurationswerte für den LUT	14
Tabelle 8: Darstellung von Verilog-Quellcode in einer physikalischen FPGA-Zelle.....	15
Tabelle 9: Liste der Unterpasse des Passes "proc"	17
Tabelle 10: Liste der Unterpasse des Passes "opt"	17
Tabelle 11: vollständiger Ablauf der Yosys-Syntheseskripte	19
Tabelle 12: gemessener Ablauf der Yosys-Syntheseskripte	21
Tabelle 13: gemessener Verlauf der Synthese eines Volladdierer in drei Beschreibung variante	24
Tabelle 14: Vergleich der physikalischen Zellen der Varianten des Volladdierers nach der GateMate-Synthese	25
Tabelle 15: Konfiguration des LUTs für den Volladdierer für die Variante A und B	28
Tabelle 16: Konfiguration des LUTs für den Volladdierer für die Variante C mit Process und case	32
Tabelle 17: Ressourcenbelegung nach dem Packing	33

Appendix

Appendix A Installation und Konfiguration von Yosys und Nextpnr

Der folgende Abschnitt erläutert die theoretischen Grundlagen zur effizienten Installation und Konfiguration der Toolchains Yosys und Nextpnr auf einem Ubuntu 22.04.5 LTS.

Appendix A.1 Yosys installieren: Die Open-Source-Synthesesuite einrichten.

1. Aktualisierung von CMake

Entfernung eines veralteten CMake-PPA (falls vorhanden) und Einbindung des offiziellen Kitware-Repositorys (GPG-Schlüssel + apt-Quelle), um eine CMake-Version ≥ 3.28 zu erhalten, die für den Build erforderlich ist.

2. Installation der Build-Abhängigkeiten

Installation der notwendigen Tools und Bibliotheken über apt:

```
sudo apt install build-essential clang bison flex tcl-dev  
libffi-dev libboost-*dev zlib1g-dev
```

3. Beschaffung des Quellcodes

Klonen des GitHub-Repositorys YosysHQ/yosys mit der Option `--recurse-submodules`, um interne Abhängigkeiten (insbesondere ABC, die Logiksynthese-Engine) mit einzubeziehen.

```
git clone --recurse-submodules  
https://github.com/YosysHQ/yosys.git
```

#Wechsle in das Verzeichnis „yosys“

```
cd yosys
```

4. Kompilierung und Installation

- Vorheriges Deaktivieren von Conda (`conda deactivate`), um Konflikte bei C++-Headern zu vermeiden

```
conda deactivate # Konflikte bei C++-Headern vermeiden
```

- Konfiguration des Builds mit `cmake -B build -DCMAKE_BUILD_TYPE=Release`

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
```

- Parallelisierte Kompilierung

```
cmake --build build -j$(nproc)
```

- Verifikation:

```
./build/yosys -V
```

- Systemweite Installation (Binärdateien landen in /usr/local/bin)

```
sudo cmake --install build
```

5. Überprüfung

Kontrolle der installierten Version (yosys -V) sowie des Installationspfads (

```
which yosys
```

```
→usr/local/bin/yosys
```

Appendix A.2 Nextpnr einrichten: Das Platzierungs- und Routing-Tool (Place & Route) konfigurieren.

Installieren Sie die Voraussetzungen für Debian/Ubuntu Linux mit apt:

```
sudo apt-get install -y libeigen3-dev libboost-all-dev python3-dev
```

Die folgenden Befehle dienen zum Herunterladen, Kompilieren und Installieren der Quelldateien, unabhängig von der verwendeten Linux-Distribution. Klonen Sie zunächst das Repository mit Git und wechseln Sie in das entsprechende Verzeichnis.

```
git clone --recurse-submodule  
https://github.com/YosysHQ/nextpnr.git  
cd nextpnr
```

Der typische Erstellungs- und Installationsprozess für Linux wird mit den folgenden beiden Befehlen aufgerufen.

```
mkdir -p build && cd build  
cmake .. -DARCH="himbaechel" \  
        -DHIMBAECHEL_UARCH="gatemate" \  
        -DHIMBAECHEL_PEPPERCORN_PATH=~/.prjpeppercorn \  
        -DCMAKE_BUILD_TYPE=Release  
make -j$(nproc)  
sudo make install
```

Falls `sudo make install` den PYTHONPATH verliert:

```
sudo env PYTHONPATH=~/.prjpeppercorn/gatemate make install
```

Der resultierende Binärname lautet **nextpnr-himbaechel** (nicht nextpnr-gatemate) — die konkrete Zielarchitektur (CCGM1A1/CCGM1A2) wird zur Laufzeit per `--device` ausgewählt.

Appendix A.3 Project Peppercorn (GateMate-Backend)

Project Peppercorn (prjpeppercorn) ist ein quelloffenes Softwareprojekt, das die Dokumentation und Werkzeuge für den **Bitstream der GateMate-FPGAs** von Cologne Chip bereitstellt. Das Projekt ermöglicht es Entwicklern, das proprietäre Dateiformat dieser FPGAs zu analysieren, zu manipulieren und eigene Bitstreams ohne die herstellereigene Software zu erzeugen.

```
git clone https://github.com/YosysHQ/prjpeppercorn.git
```

Peppercorn enthält zwei Teile:

- **libgm/** — C++-Tools (gmpack, gmunpack für Bitstream-Erzeugung), kompiliert via CMake
- **gatemate/chip.py** — reines Python-Modul, das von nextpnr's Chipdatenbank-Generator (arch_gen.py) importiert wird

```
cd prjpeppercorn/libgm
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

Wichtige Falle: arch_gen.py benötigt `import chip`, was auf `gatemate/chip.py` verweist — **nicht** auf ein kompiliertes Modul aus libgm. Ohne korrekten PYTHONPATH schlägt der nextpnr-Build mit `ModuleNotFoundError: No module named 'chip'` fehl.

```
export PYTHONPATH=~/.prjpeppercorn/gatemate:$PYTHONPATH
# dauerhaft:
echo 'export PYTHONPATH=~/.prjpeppercorn/gatemate:$PYTHONPATH' >>
~/.bashrc
```

```
nextpnr-himbaechel -version
```

Which nextpnr-himbaechel

Appendix B GateMate FPGA Open-Source-Toolchain &Workflow [37]

Dieser Anhang beschreibt den vollständigen, skriptbasierten Entwicklungs- und Implementierungs-Workflow für GateMate FPGAs auf Basis der **OSS CAD Suite**. Er umfasst die Synthese mit Yosys, das Place-and-Route mit nextpnr-himbaechel, die Bitstream-Generierung mittels gmpack sowie die Programmierung über openFPGALoader.

Appendix B1 Komponenten der OSS CAD Suite

Die Open-Source-Toolchain fasst Synthese-, Implementierungs- und Verifikationswerkzeuge in einer vorkompilierten Umgebung zusammen:

- **Kern-Toolchain:**
 - yosys: Framework zur RTL-Synthese für Verilog, SystemVerilog und VHDL.
 - nextpnr-himbaechel: Timing-getriebenes Place-and-Route-Werkzeug für die GateMate-Architektur.
 - gmpack: Pack-Utility zur Erzeugung des proprietären GateMate-Bitstreams (.bit) aus der PnR-Netzliste.
 - openFPGALoader: Universelles Programmierwerkzeug zur Konfiguration via JTAG, SPI oder SPI-Flash.
- **Simulations- und Analysewerkzeuge**
 - ghdl: VHDL-Simulator (Linux/macOS).
 - iverilog: IEEE-1364-konformer Verilog-Compiler und Simulator.
 - verilator: Zyklengenauer C++/SystemC-Simulator für Verilog/SystemVerilog.
 - cocotb: Python-basiertes Framework für Coroutine-Cosimulationen.
 - gtkwave / surfer: Grafische Signalverlaufs-Viewer (Waveform-Viewer).

Appendix B.2 Synthese mit Yosys (synth_gatemate)

Die Übersetzung von Verilog/SystemVerilog-Quellen in eine auf GateMate-Primitive gemappte Netzliste erfolgt über das Frontend `read_verilog` und den Zielarchitektur-Pass `synth_gatemate`.

```
yosys \  
-ql synthesis.log \  
-p '  
  read_verilog -sv <sources>;  
  synth_gatemate \  
    -top <topmodule> \  
    -luttree \  
    -nomx8;  
  write_json netlist.json;  
  write_verilog netlist_synth.v;
```

Parameterübersicht: synth_gatemate

Parameter	Typ	Beschreibung
-top <module>	Obligatorisch	Name des Top-Level-Moduls.
-luttree	Obligatorisch	Aktiviert die Abbildung auf die LUT-Tree-Struktur (CC_L2T4 / CC_L2T5).
-nomx8	Obligatorisch	Verbietet MUX8-Primitiven (GateMate besitzt nativ nur MUX2 und MUX4).
-vlog <file>	Optional	Schreibt die synthetisierte Netzliste als Verilog-Datei aus.

-json <file>	Optional	Schreibt die Netzliste direkt im JSON-Format für nextpnr.
-run <from>:<to>	Optional	Führt gezielt Abschnitte zwischen Labels aus (z. B. begin:map_cells).
-noflatten	Optional	Verhindert das Auflösen der Modulhierarchie vor der Synthese.
-nobram	Optional	Unterbindet die Inferenz von CC_BRAM_20K / CC_BRAM_40K-Zellen.
-noaddf	Optional	Deaktiviert die Nutzung dedizierter Volladdierer (CC_ADDF).
-nomult	Optional	Deaktiviert die Inferenz harter Multiplizierer-Strukturen (CC_MULT).
-nomx4	Optional	Verbietet 4:1-Multiplexer (CC_MX4).
-noiopad	Optional	Deaktiviert automatisches Einfügen von I/O-Pads (z. B. für Out-of-Context-Synthesen).
-noclbuf	Optional	Deaktiviert das automatische Einfügen von globalen Taktpuffern.

Hinweis zur Verifikation: Vor der Weiterverarbeitung sollte das Syntheseprotokoll stets auf unabsichtlich inferierte Latches (\$dlatch) geprüft und der Ressourcenverbrauch mittels des Befehls stat innerhalb von Yosys kontrolliert werden.

Appendix B.3 Implementierung: Place & Route mit nextpnr-himbaechel

Das generierte JSON-Netzlistenformat wird mit nextpnr-himbaechel auf die physikalische Matrix (CPes, BRAM, I/O-Bänke) abgebildet.

Bash

```
nextpnr-himbaechel \
  -l pnr.log \
  --device=CCGM1A1 \
  --json netlist.json \
  -o ccf=constraints.ccf \
  -o out=impl.txt \
  --sdc timing.sdc \
  --router router2
```

Parameterübersicht: nextpnr-himbaechel

Parameter / Option	Wertebereich	Funktion
--device <device>	CCGM1A1, CCGM1A2	Wählt das Zielbauelement (A1: 20.480 CPEs).
--router <alg>	router1, router2	Routing-Algorithmus (Standard & empfohlen: router2).
--placer <alg>	heap, sa, static	Platzierungs-Algorithmus (Standard: heap).
--sdc <file>	Dateipfad	Bindet Synopsys Design Constraints (Takte, I/O-Delays) ein.
--sdf <file>	Dateipfad	Erzeugt Standard Delay Format-Datei für Back-Annotated-Simulationen.
--gui	Flag	Startet die grafische Oberfläche zur Pfad- und Belegungsinspektion.
--timing-allow-fail	Flag	Bricht den Lauf bei Nichterreichen des Timing-Closures nicht ab.

-o ccf=<file>	Dateipfad	Cologne Constraint File: Definiert Pinbelegung und I/O-Standards.
-o fpga_mode=<mode>	lowpower, economy, speed	Setzt den Betriebsmodus des GateMate-Kerns.
-o time_mode=<mode>	best, typical, worst	Wählt die Corner-Bedingung für die statische Timing-Analyse.
-o strategy=<strat>	mirror, full, clk1	Multi-Die-Taktverteilungsstrategie.
-o force_die=<ID>	z. B. 1A, 1B	Erzwingt die Platzierung auf einem spezifischen Die.
-o no-cpe-cp	Flag	Verhindert CP-Leitungsdurchgänge durch CPE-Zellen.
-o no-bridges	Flag	Unterbindet die Nutzung von CPEs im Brückenmodus.
-o allow-unconstrained	Flag	Erlaubt ungebundene I/O-Pins (nur für Testläufe empfohlen).

Appendix B.4 Bitstream-Generierung mit gmpack

Nach erfolgreichem PnR wird die Zwischendatei (impl.txt) in die binäre Konfigurationsdatei konvertiert:

```
gmpack \
  --crcmode check \
  --spimode single \
  impl.txt \
  bitstream.bit
```

- **--spimode <single|dual|quad>**: Definiert die Lese-Geschwindigkeit aus dem externen SPI-Flash.
- **--crcmode <check|ignore|unused>**: Steuert das Verhalten bei fehlerhafter Prüfsumme.
- **--reconfig**: Aktiviert die integrierte Rekonfigurationsschnittstelle.
- **--bootaddr <addr>**: Startadresse für Multi-Boot-/Fallback-Bitstreams im Flash.
- **--background**: Ermöglicht Hintergrund-Rekonfiguration im Flash-Modus.

Appendix B.5 Hardware-Konfiguration mit openFPGALoader

Die Programmierung des GateMate FPGAs erfolgt flexibel flüchtig (SRAM) oder persistent (SPI-Flash):

→ **Flüchtig über JTAG (Schneller Debug-Modus):**

```
openFPGALoader -b gatemate_evb_jtag --freq 6M bitstream.bit
```

→ **Flüchtig über SPI:**

```
openFPGALoader -b gatemate_evb_spi --freq 6M -m bitstream.bit
```

→ **Persistent im On-Board-SPI-Flash:**

```
openFPGALoader -b gatemate_evb_jtag -f bitstream.bit
```

Appendix C Quellen code

Appendix C.1 Voll_addierer ohne Process

```
module Full_adder(a, b, cin, sum, co);  
    input a, b, cin;  
    output sum, co;  
  
    assign sum = a ^ b ^ cin;
```

```
    assign co = (a & b) | (a & cin) | (b & cin);  
endmodule
```

Appendix C.2 Voll_addierer mit Process

```
module Full_adder_process(  
    input  a, b, cin,  
    output reg sum,  
    output reg co  
);  
    always @(*) begin  
        sum = a ^ b ^ cin;  
        co  = (a & b) | (a & cin) | (b & cin);  
    end  
endmodule
```

Appendix C.3 Voll_addierer mit Process und case

```
module Full_adder_process_case (  
    input  wire a,  
    input  wire b,  
    input  wire cin,  
    output reg  sum,  
    output reg  cout  
);  
    always @(*) begin  
        case ({a, b, cin})  
            3'b000: begin sum = 1'b0; cout = 1'b0; end  
            3'b001: begin sum = 1'b1; cout = 1'b0; end  
            3'b010: begin sum = 1'b1; cout = 1'b0; end  
            3'b011: begin sum = 1'b0; cout = 1'b1; end  
        endcase  
    end  
endmodule
```

```
        3'b100: begin sum = 1'b1; cout = 1'b0; end
        3'b101: begin sum = 1'b0; cout = 1'b1; end
        3'b110: begin sum = 1'b0; cout = 1'b1; end
        3'b111: begin sum = 1'b1; cout = 1'b1; end
    endcase
end
endmodule
```

Appendix C.4 Yosyskript für die Analyse der Zwischenschritte

```
# =====
# GateMate.yys -- Step-by-step GateMate flow, on Full_adder.v
#
# Each step produces three files:
#   GateMate/NN_label.il      Complete RTLIL, the only lossless export
#   GateMate/NN_label.stat    Cell count
#   Bilder/NN_label.png       Diagram, via Graphviz
#
# Prerequisites: mkdir -p GateMate Bilder (and Graphviz installed)
# Run:          yosys GateMate.yys
#
# Note 1: 'show' writes a file instead of opening a window thanks to
# -format png -prefix. Do NOT add -viewer none: that combination
# triggers an assertion error.
#
# Note 2: the final argument 'A:top' selects the module carrying the
# \top attribute, whatever its name. This keeps the diagram free of
# the library modules loaded by the 'begin' step, and makes the script
# immune to capitalisation mistakes. It requires 'hierarchy -top' to
# have run first, which is why the line below is explicit.
#
# If 'hierarchy' fails, the module name is wrong. Module names are
# case sensitive. List the real ones with: yosys -p "read_verilog Full_adder.v; ls"
# =====
```

```
read_verilog Full_adder.v
hierarchy -check -top Full_adder
write_rtlil GateMate/00_read_verilog.il
tee -o      GateMate/00_read_verilog.stat stat
show -format png -prefix Bilder/00_read_verilog A:top

# --- (read) -----
synth_gatemate -top Full_adder -luttree -nomx8 -run :begin
write_rtlil GateMate/01_hierarchy.il
tee -o      GateMate/01_hierarchy.stat stat
show -format png -prefix Bilder/01_hierarchy A:top

# --- begin -----
synth_gatemate -top Full_adder -luttree -nomx8 -run begin:prepare
write_rtlil GateMate/02_begin.il
tee -o      GateMate/02_begin.stat stat
show -format png -prefix Bilder/02_begin A:top

# --- prepare -----
synth_gatemate -top Full_adder -luttree -nomx8 -run prepare:map_mult
write_rtlil GateMate/03_prepare.il
tee -o      GateMate/03_prepare.stat stat
show -format png -prefix Bilder/03_prepare A:top

# --- map_mult -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_mult:coarse
write_rtlil GateMate/04_map_mult.il
tee -o      GateMate/04_map_mult.stat stat
show -format png -prefix Bilder/04_map_mult A:top

# --- coarse -----
synth_gatemate -top Full_adder -luttree -nomx8 -run coarse:map_bram
write_rtlil GateMate/05_coarse.il
tee -o      GateMate/05_coarse.stat stat
show -format png -prefix Bilder/05_coarse A:top
```

```
# --- map_bram -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_bram:map_ffram
write_rtlil GateMate/06_map_bram.il
tee -o GateMate/06_map_bram.stat stat
show -format png -prefix Bilder/06_map_bram A:top

# --- map_ffram -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_ffram:map_gates
write_rtlil GateMate/07_map_ffram.il
tee -o GateMate/07_map_ffram.stat stat
show -format png -prefix Bilder/07_map_ffram A:top

# --- map_gates -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_gates:map_io
write_rtlil GateMate/08_map_gates.il
tee -o GateMate/08_map_gates.stat stat
show -format png -prefix Bilder/08_map_gates A:top

# --- map_io -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_io:map_regs
write_rtlil GateMate/09_map_io.il
tee -o GateMate/09_map_io.stat stat
show -format png -prefix Bilder/09_map_io A:top

# --- map_regs -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_regs:map_muxs
write_rtlil GateMate/10_map_regs.il
tee -o GateMate/10_map_regs.stat stat
show -format png -prefix Bilder/10_map_regs A:top

# --- map_muxs -----
synth_gatemate -top Full_adder -luttree -nomx8 -run map_muxs:map_luts
write_rtlil GateMate/11_map_muxs.il
tee -o GateMate/11_map_muxs.stat stat
show -format png -prefix Bilder/11_map_muxs A:top
```

```
# --- map_luts -----
synth_gatmate -top Full_adder -luttree -nomx8 -run map_luts:map_cells
write_rtlil GateMate/12_map_luts.il
tee -o GateMate/12_map_luts.stat stat
show -format png -prefix Bilder/12_map_luts A:top

# --- map_cells -----
synth_gatmate -top Full_adder -luttree -nomx8 -run map_cells:map_bufg
write_rtlil GateMate/13_map_cells.il
tee -o GateMate/13_map_cells.stat stat
show -format png -prefix Bilder/13_map_cells A:top

# --- map_bufg -----
synth_gatmate -top Full_adder -luttree -nomx8 -run map_bufg:check
write_rtlil GateMate/14_map_bufg.il
tee -o GateMate/14_map_bufg.stat stat
show -format png -prefix Bilder/14_map_bufg A:top

# --- check -----
synth_gatmate -top Full_adder -luttree -nomx8 -run check:
write_rtlil GateMate/15_check.il
tee -o GateMate/15_check.stat stat
show -format png -prefix Bilder/15_check A:top

# --- final netlist for nextpnr -----
write_json Full_adder.json
```

Appendix D Simulation der Variante A/B und C

Appendix D.1 Simulation der Synthesenergebnisse der Variante A/B mit logic.ly

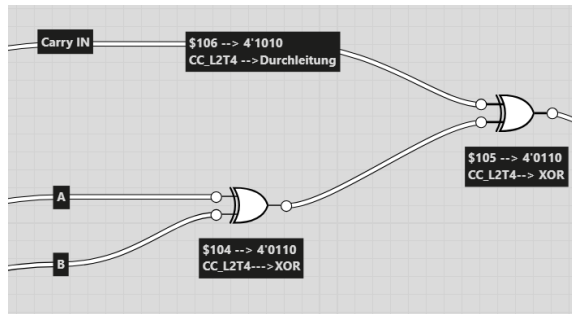


Abbildung 18: Konfiguration der LUTs für die Funktion $(a \wedge b) \& cin$

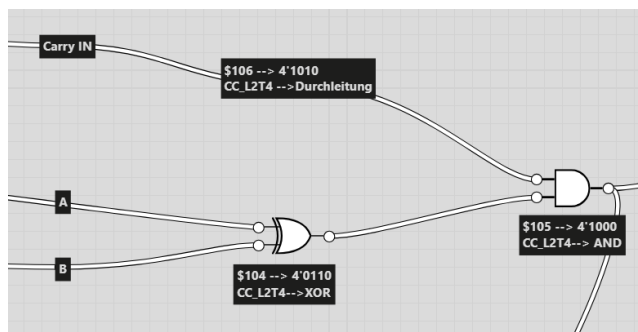


Abbildung 19: Konfiguration der LUTs für die Funktion $a \wedge b \wedge c$

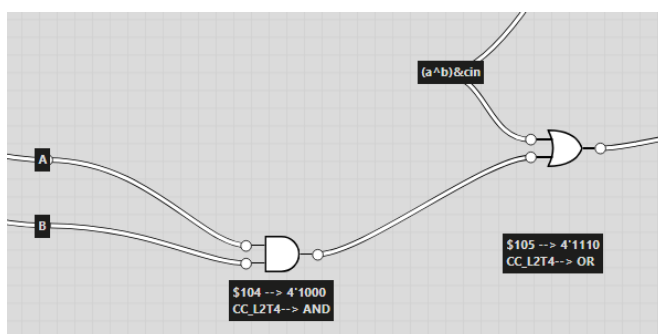


Abbildung 20: Konfiguration der LUTs für die Funktion $(a \& b) \mid ((a \wedge b) \& cin) \rightarrow cout$

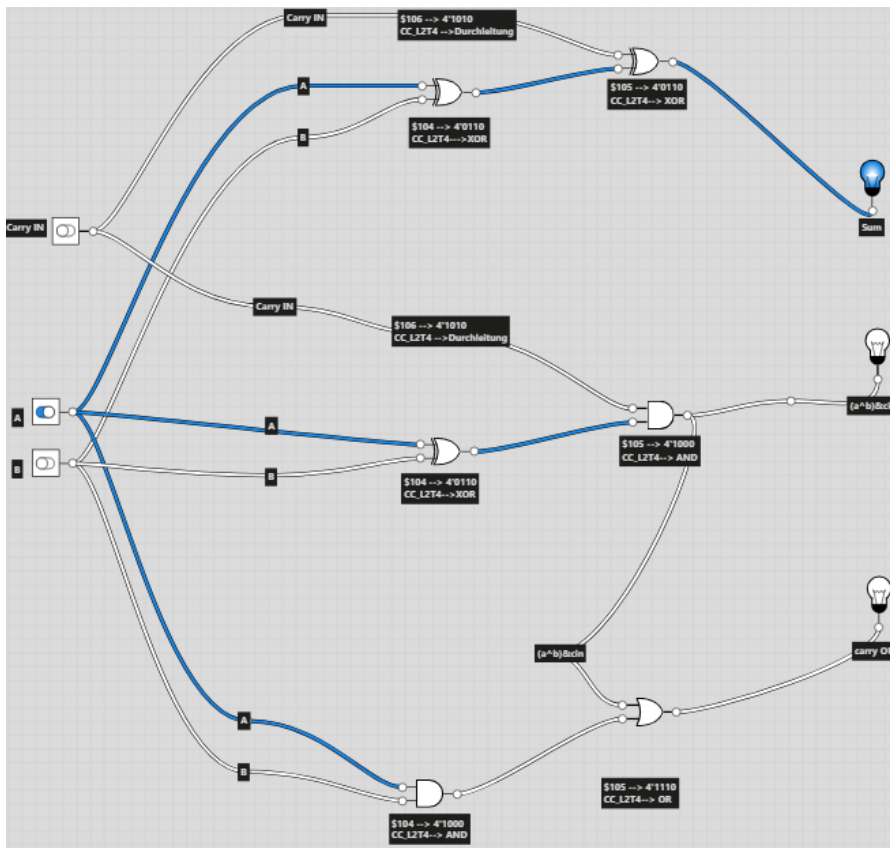


Abbildung 21: Simulation der Addition $a(1) + b(0)$, cin (0) mit der von Yosys konfigurierte LUT-Tabelle der Variante A und B

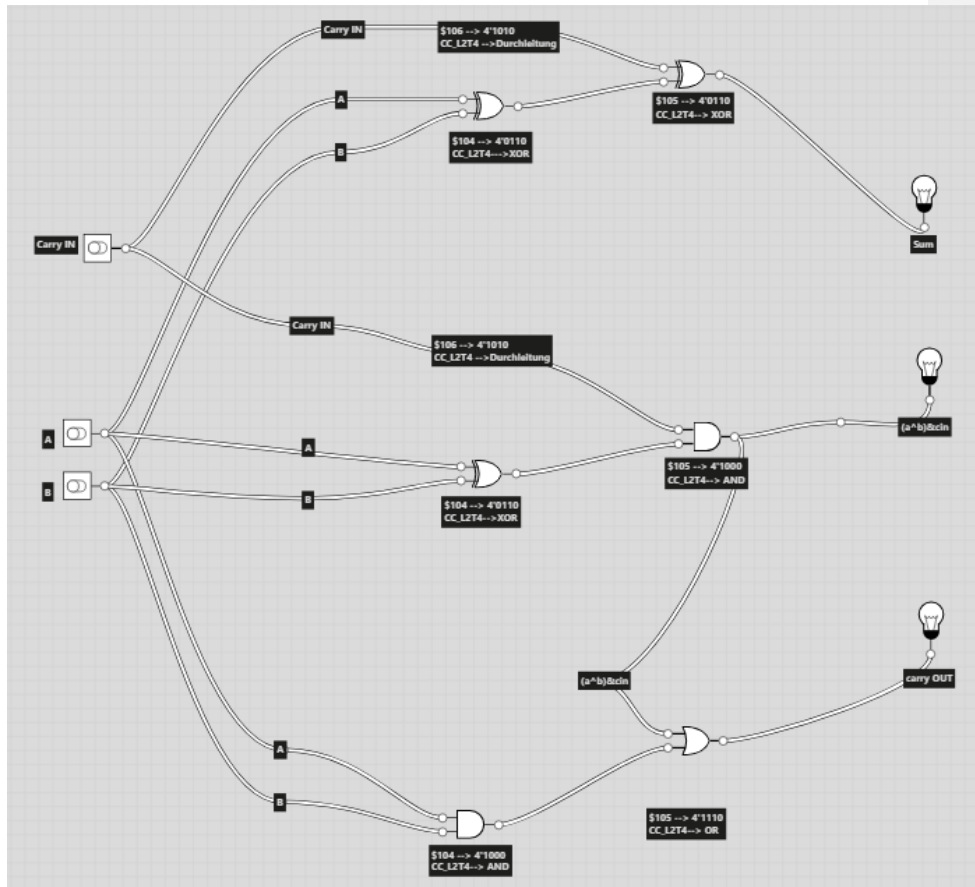


Abbildung 22: Simulation der Addition $a(0) + b(1)$, cin (0) mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B

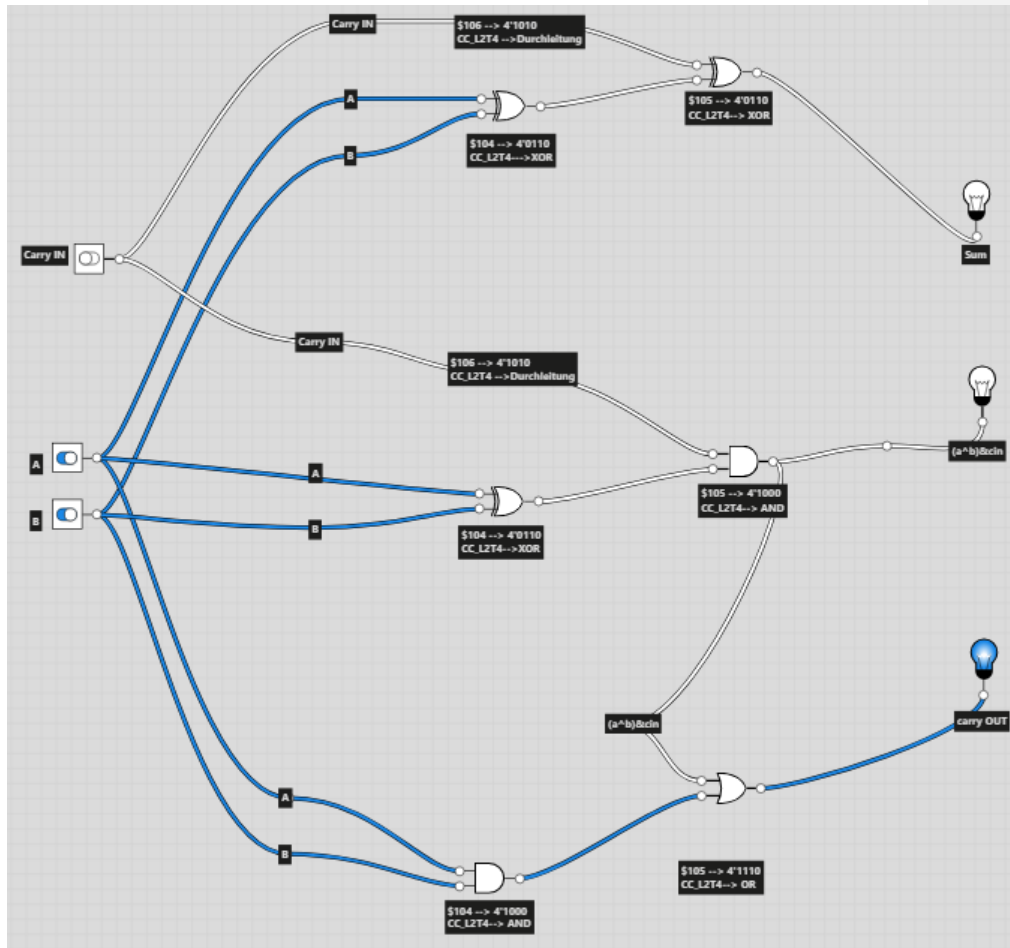


Abbildung 23: Simulation der Addition $a(1) + b(1)$, cin (0) mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B

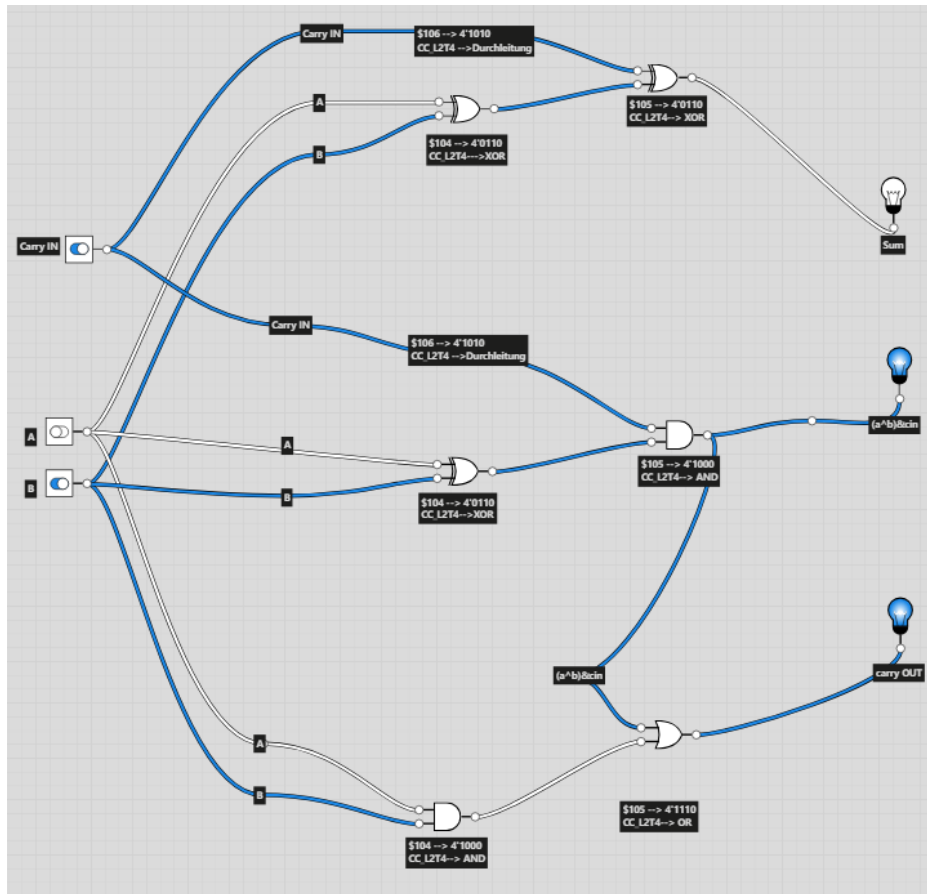


Abbildung 24: Simulation der Addition $a(0) + b(1)$, cin (1) mit der von Yosys konfigurierten LUT-Tabelle der Variante A und B

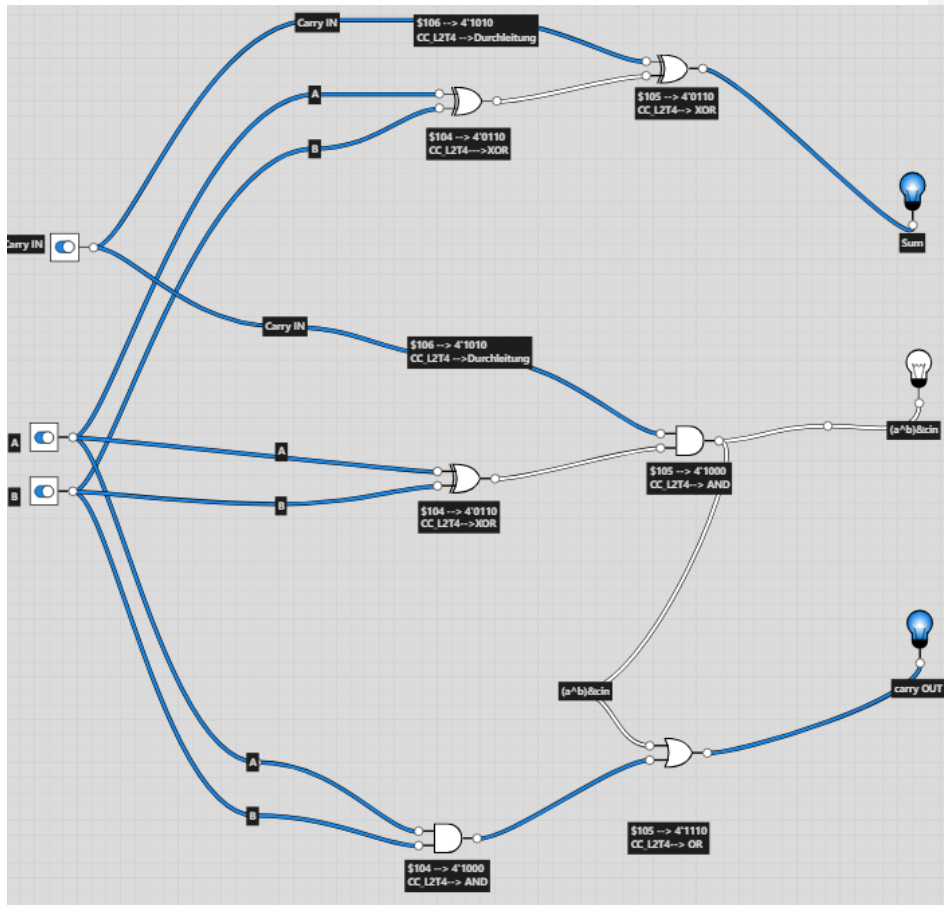


Abbildung 25: Simulation der Addition $a(1) + b(1)$, cin (1) mit der von Yosys konfigurierter LUT-Tabelle der Variante A und B

Appendix D.2 Simulation der Synthesenergebnisse der Variante C mit simulator.io [9]

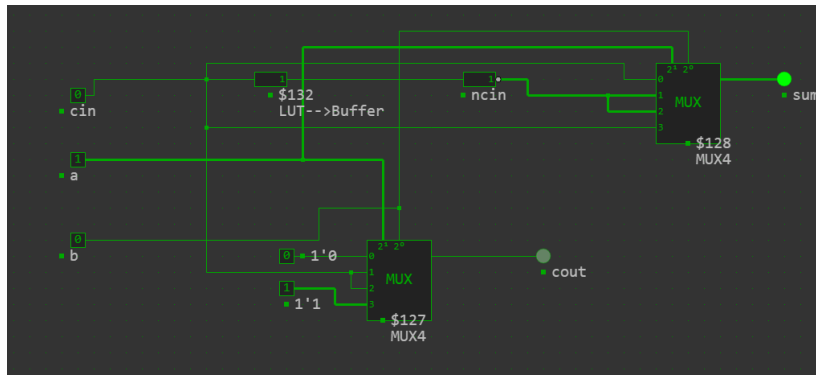


Abbildung 26: Simulation der Addition $a(1) + b(0)$, $cin(0)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante C

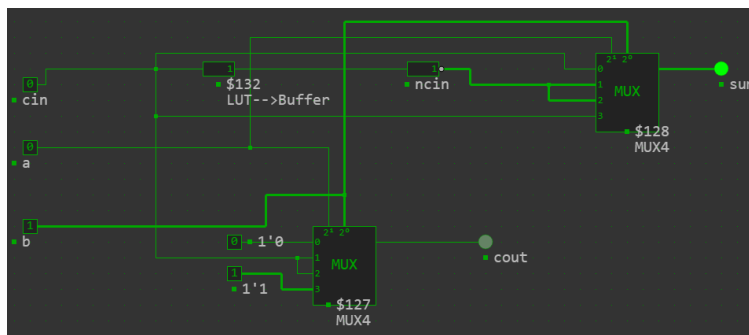


Abbildung 27: Simulation der Addition $a(0) + b(1)$, $cin(0)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante C

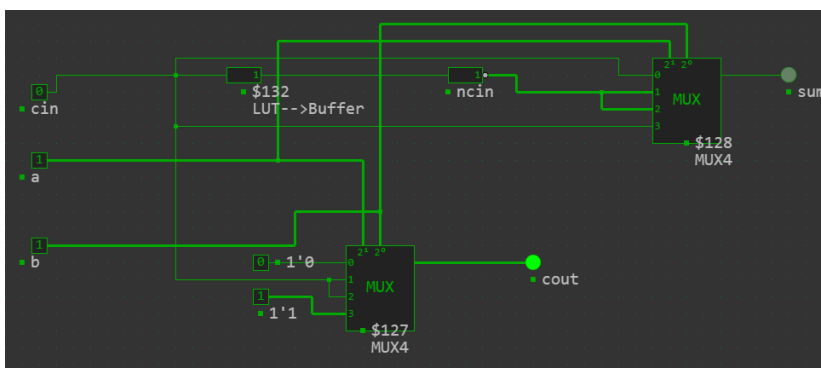


Abbildung 28: Simulation der Addition $a(1) + b(1)$, $cin(0)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante C

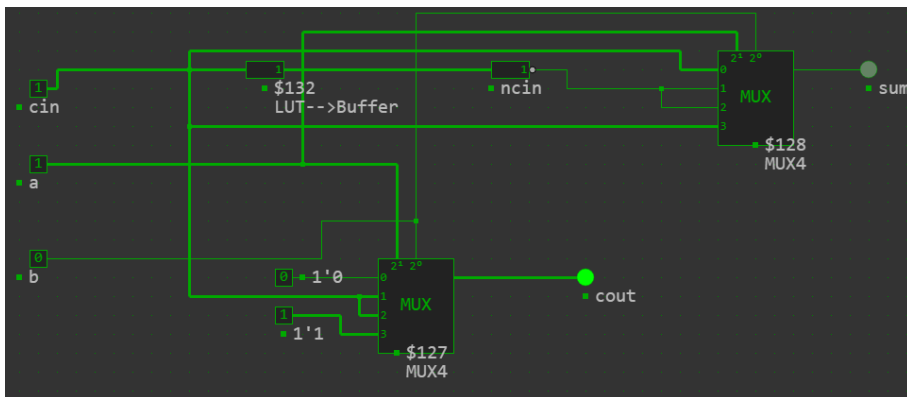


Abbildung 29: Simulation der Addition $a(1) + b(0)$, $cin(1)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante C

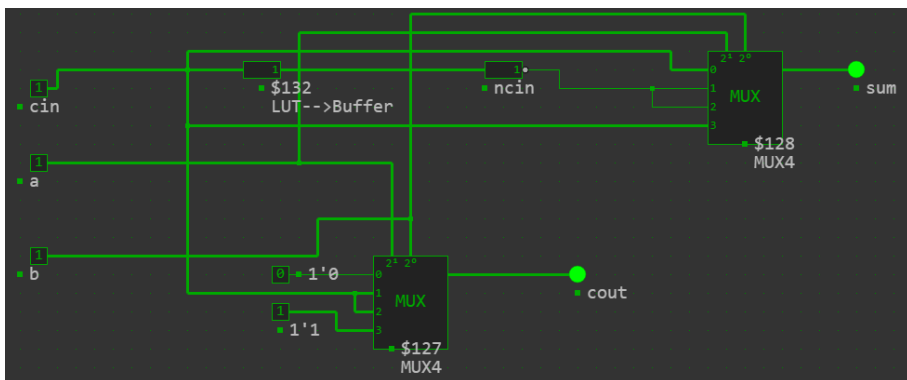


Abbildung 30: Simulation der Addition $a(1) + b(1)$, $cin(1)$ mit der von Yosys konfigurierter LUT-Tabelle der Variante C